Essentials of Web Programming

UNIT - IV

TypeScript:

JavaScript was created in 1995 and became one of the major languages for web programming. As web applications became larger and more complex, JavaScript's dynamic typing sometimes made large programs more difficult to develop and maintain because many type-related mistakes were discovered only while the program was running.

Microsoft began developing TypeScript to provide stronger development-time checking while keeping compatibility with JavaScript. TypeScript was publicly introduced by Microsoft in 2012, with Anders Hejlsberg serving as its lead architect. Hejlsberg is also known for his major role in the development of languages such as C#.

The first public TypeScript release was in 2012, and TypeScript 1.0 was released in 2014. TypeScript has continued to evolve while remaining closely connected to the JavaScript and ECMAScript ecosystem.

The historical development can be represented as:

1995

JavaScript Created

↓

Web Applications Grow

↓

Larger JavaScript Programs

↓

Need for Better Type Checking

↓

2012

TypeScript Introduced by Microsoft

↓

2014

TypeScript 1.0

↓

Continued Development

↓

Modern TypeScript

Important Facts

Developed by → Microsoft

Publicly introduced → 2012

Lead architect → Anders Hejlsberg

TypeScript 1.0 → 2014

Main purpose → Improve large-scale JavaScript development

Relationship → TypeScript extends JavaScript

Note: TypeScript was not created to replace JavaScript. It was created to make JavaScript development more structured and type-safe, especially for larger applications.

4.1 Introduction to TypeScript and Installation

4.1.1 Introduction

JavaScript is one of the main programming languages used to develop interactive web applications. It allows developers to create variables, functions, objects, classes, and many other program components. However, JavaScript is dynamically typed, which means that the type of a variable is generally determined from the value assigned to it and can change during program execution.

For small programs, this flexibility is convenient. In larger applications, however, an incorrect type of value may produce errors that are difficult to identify.

TypeScript was introduced to provide additional type-checking features while retaining the basic programming capabilities of JavaScript.

4.1.2 TypeScript

Definition: TypeScript is an open-source programming language developed and maintained by Microsoft that extends JavaScript by adding static type-checking and additional development features. It allows programmers to describe the expected types of variables, parameters, objects, and other program elements so that many errors can be detected during development. TypeScript source code is normally converted into JavaScript before execution. It is particularly useful for developing large, structured, and maintainable JavaScript applications.

JavaScript

+

Type Checking

↓

TypeScript

TypeScript does not replace JavaScript in the browser.

The general process is:

TypeScript Program

↓

TypeScript Compiler

↓

JavaScript Program

↓

Browser

Scenario

Consider an online shopping website where productPrice should always contain a numeric value. TypeScript can help ensure that the developer does not accidentally assign a text value to that variable.

4.1.3 Need for TypeScript

JavaScript allows the following:

```
let productPrice = 50000;
productPrice = "Fifty Thousand";
```

The same variable initially contains a number and later contains a string.

In a large shopping website, this may cause incorrect calculations.

For example:

```
let productPrice = "Fifty Thousand";
let quantity = 2;
let total = productPrice * quantity;
```

The intended numerical calculation cannot be performed correctly.

TypeScript allows the expected type to be specified:

```
let productPrice: number = 50000;
```

If the programmer later writes:

```
productPrice = "Fifty Thousand";
```

TypeScript reports a type error during type checking.

Need can therefore be represented as:

Large JavaScript Application

↓

Possible Type Mistakes

↓

Difficult Error Detection

↓

TypeScript Type Checking

↓

Earlier Error Detection

Scenario

A college admission system may contain:

Student Name → Text

Student Age → Number

Admission Open → True / False

TypeScript allows these expected value types to be clearly specified, reducing accidental misuse of data.

4.1.4 TypeScript and JavaScript

Definition: TypeScript is based on JavaScript and extends the JavaScript language rather than replacing it. Existing JavaScript concepts such as variables, functions, arrays, objects, classes, and modules continue to be available in TypeScript. TypeScript adds development-time type information and related language features. After compilation, the generated JavaScript can be executed in environments that support JavaScript.

JavaScript Example

```
let websiteName = "College Portal";
```

TypeScript Example

```
let websiteName: string = "College Portal";
```

Here:

websiteName

→ Variable

string

→ Expected type

College Portal

→ Value

Important

JavaScript

↑

TypeScript extends JavaScript

↓

TypeScript is compiled

↓

JavaScript

Scenario

A development team already familiar with JavaScript can continue using familiar JavaScript concepts while gradually adding TypeScript type information to a project.

4.1.5 TypeScript File

A TypeScript source file normally uses the extension:

.ts

Example:

app.ts

A JavaScript file normally uses:

.js

Example:

app.js

Therefore:

TypeScript → .ts

JavaScript → .js

Scenario

A shopping application may contain a TypeScript source file:

product.ts

After compilation, JavaScript code can be generated for execution.

4.1.6 TypeScript Compiler

Definition: The TypeScript compiler is a software tool that checks TypeScript source code and converts it into JavaScript code. The compiler is commonly accessed using the command tsc. During compilation, TypeScript can report type-related errors found in the source program. The generated JavaScript can then be executed by a browser or another JavaScript runtime.

Basic Command

tsc app.ts

If the source file is:

app.ts

the compilation process can generate:

app.js

Working

app.ts

↓

tsc

↓

Type Checking

↓

app.js

Website Scenario

A developer writes a college portal program in:

portal.ts

The TypeScript compiler checks the program and generates JavaScript that can be used by the web application.

4.1.7 Requirements for Installation

A common beginner setup uses:

Node.js

↓

npm

↓

TypeScript

Node.js provides a JavaScript runtime environment and commonly includes npm, the Node package manager.

Before installing TypeScript, the student can verify Node.js using:

node --version

and npm using:

npm --version

If version numbers are displayed, the tools are available.

4.1.8 Installing TypeScript

TypeScript can be installed through npm.

Command

npm install -g typescript

Here:

npm

→ Node package manager

install

→ Installs a package

-g

→ Installs it globally

typescript

→ Package being installed

After installation, verify the TypeScript compiler using:

tsc --version

A displayed version number confirms that the compiler is available.

4.1.9 Creating the First TypeScript Program

Create a file named:

app.ts

Write:

```
let websiteName: string =
        "College Portal";

console.log(websiteName);
```

This program stores the name of a website and displays it.

4.1.10 Compiling a TypeScript Program

Open the terminal in the folder containing app.ts.

Execute:

tsc app.ts

The compiler processes:

app.ts

and produces JavaScript such as:

app.js

The generated JavaScript will be similar to:

```
let websiteName =
        "College Portal";


console.log(websiteName);
```

Notice that the TypeScript type annotation:

: string

does not appear in the generated JavaScript.

The type information is primarily used by TypeScript during development and type checking.

4.1.11 Executing the Generated JavaScript

After compilation, the generated JavaScript can be executed.

For example, using Node.js:

node app.js

Output

College Portal

The complete process is:

Write

app.ts

↓

Compile

tsc app.ts

↓

Generate

app.js

↓

Execute

node app.js

↓

Output

College Portal

4.1.12 Working Principle of TypeScript

The working principle can be represented as:

Developer Writes

TypeScript Code

↓

TypeScript Compiler

↓

Checks Types

↓

Reports Type Errors

↓

Generates JavaScript

↓

JavaScript Executes

An important point is:

Browsers normally execute JavaScript, not TypeScript source code directly.

Therefore, TypeScript is generally converted into JavaScript before execution.

4.1.13 Scenario - College Portal

Consider a college website that stores the number of available admission seats.

In TypeScript:

let availableSeats: number = 60;

The developer accidentally writes:

availableSeats = "Sixty";

TypeScript identifies that a string is being assigned where a number is expected.

The intended flow becomes:

Developer Writes Code

↓

TypeScript Checks Type

↓

Incorrect Type Found

↓

Error Reported

↓

Developer Corrects Code

This helps identify the mistake before it becomes part of the running application.

4.1.14 Program

Create a file named:

website.ts

Program

```
let websiteName: string =
        "ABC College Portal";


let availableSeats: number = 60;
let admissionOpen: boolean = true;
console.log(
        "Website: " + websiteName
);
console.log(
        "Available Seats: " +
        availableSeats
```

```
);

console.log(
        "Admission Open: " +
        admissionOpen
);
```

Compile using:

```
tsc website.ts
```

Then execute the generated JavaScript:

```
node website.js
```

Output

```
Website: ABC College Portal
Available Seats: 60
Admission Open: true
```

The data types shown here are only introductory examples. Variables and Data Types are studied in detail in Topic 4.2.

4.1.15 Program Explanation

The statement:

```
let websiteName: string =
        "ABC College Portal";
```

stores textual information representing the website name.

The statement:

```
let availableSeats: number = 60;
```

stores a numerical value.

The statement:

```
let admissionOpen: boolean = true;
```

stores a true-or-false value.

The console.log() statements display the stored information.

The file is compiled using:

```
tsc website.ts
```

and the generated JavaScript can then be executed.

4.1.16 Important Points

TypeScript extends JavaScript.

A TypeScript source file commonly uses:

.ts

TypeScript supports static type checking.

The TypeScript compiler command is:

tsc

TypeScript is commonly installed using:

npm install -g typescript

The basic process is:

TypeScript

→ Check

→ Compile

→ JavaScript

→ Execute

4.1.17 Scenario-Based Questions

Question 1

A shopping website wants productPrice to contain only numerical values. Which language feature can help identify an accidental string assignment during development?

Answer: TypeScript's static type checking can identify the incompatible assignment.

Question 2

A student has written a program named college.ts. Which command can be used to compile it?

Answer: tsc college.ts

Question 3

Which extension is normally used for a TypeScript source file?

Answer: .ts

Question 4

A developer wants to verify whether the TypeScript compiler is installed. Which command can be used?

Answer: tsc --version

Question 5

Can a browser normally execute the TypeScript source file directly?

Answer: TypeScript source code is normally compiled into JavaScript, and the generated JavaScript is executed by the browser.

4.1.18 Important Questions

1. What is TypeScript?

TypeScript is an open-source language developed and maintained by Microsoft that extends JavaScript with static type checking and additional development features. TypeScript programs are normally compiled into JavaScript before execution.

2. Why is TypeScript required?

TypeScript helps developers detect many type-related programming errors during development. It also makes program structures and expected data types clearer, which is useful in larger applications.

3. What is the extension of a TypeScript file?

A TypeScript source file normally uses the .ts extension.

4. What is tsc?

tsc is the TypeScript compiler command. It checks TypeScript source code and generates JavaScript.

5. How can TypeScript be installed using npm?

npm install -g typescript

6. How can the TypeScript installation be verified?

tsc --version

7. What is the relationship between TypeScript and JavaScript?

TypeScript extends JavaScript by adding type-checking and related language features. TypeScript code is generally compiled into JavaScript for execution.

4.1.19 Summary

TypeScript is a programming language that extends JavaScript by adding static type-checking and other development features. It helps identify many type-related errors while a program is being developed.

TypeScript files normally use the .ts extension. The TypeScript compiler is commonly accessed through the tsc command. TypeScript can be installed through npm, and a TypeScript source file can be compiled into JavaScript before execution.

The fundamental process is:

Write TypeScript

↓

Check Types

↓

Compile using tsc

↓

Generate JavaScript

↓

Execute JavaScript

4.2 Variables, Data Types and Functions

4.2.1 Introduction

Variables, data types, and functions are fundamental elements of TypeScript programming. Variables are used to store information, data types specify what kind of information can be stored, and functions organize reusable operations.

TypeScript extends the familiar JavaScript syntax by allowing programmers to explicitly specify the expected type of a variable, function parameter, and return value. This type information helps the compiler identify many incorrect assignments and function calls before the program is executed.

For example, an online shopping website may store a product name as text, product price as a number, and product availability as a Boolean value. TypeScript allows these expected types to be clearly specified.

4.2.2 Variables in TypeScript

Definition: A variable is a named storage location used to hold a value that may be accessed and used by a program. In TypeScript, variables can be declared using JavaScript keywords such as let and const, and an expected data type can be specified using a type annotation. TypeScript checks whether the values assigned to the variable are compatible with its expected type. Variables are used to represent information such as names, prices, quantities, marks, and status values.

Basic Syntax

```
let variableName: dataType = value;
```

Example

```
let productPrice: number = 50000;
```

Here:

productPrice → Variable name

number → Data type

50000 → Initial value

Scenario

An e-commerce website needs to store the current quantity of a product in a shopping cart.

```
let cartQuantity: number = 2;
```

The variable cartQuantity is expected to contain numerical values.

4.2.3 The let Keyword

Definition: The let keyword declares a block-scoped variable whose value can be reassigned after its initial declaration. It is useful when the stored value is expected to change while the program is running. TypeScript can also associate an explicit or inferred type with a variable declared using let. Modern TypeScript programs generally prefer let over var when reassignment is required.

Syntax

```
let variableName: dataType = value;
```

Example

```
let availableSeats: number = 50;
availableSeats = 45;
```

The value changes from:

50

↓

45

but its type remains compatible with number.

Incorrect Assignment

```
let availableSeats: number = 50;
availableSeats = "Forty Five";
```

TypeScript reports an error because a string cannot be assigned to a variable declared as a number.

Scenario

A college admission website initially has:

let availableSeats: number = 60;

When ten seats are filled:

availableSeats = 50;

Since the value changes, let is appropriate.

4.2.4 The const Keyword

Definition: The const keyword declares a block-scoped variable whose binding cannot be reassigned after initialization. A value must normally be provided when a const variable is declared. It is suitable for values that should continue referring to the same value or object throughout that scope. Modern TypeScript and JavaScript programs commonly use const by default when reassignment is not required.

Syntax

const variableName: dataType = value;

Example

const collegeName: string =

"ABC Engineering College";

The following reassignment is not allowed:

collegeName =

"XYZ Engineering College";

Scenario

The name of a website may remain unchanged while the page is running.

const websiteName: string =

"Online Learning Portal";

Therefore, const is suitable.

Simple Difference

let

→ Value may be reassigned

const

→ Variable binding cannot be reassigned

4.2.5 Type Annotation

Definition: A type annotation explicitly specifies the expected data type of a variable, parameter, or other TypeScript program element. It is written after the identifier using a colon followed by the type name. Type annotations allow the TypeScript compiler to check whether assigned or supplied values are compatible with the declared type. They also make the intended use of program data clearer to developers.

Syntax

variableName: dataType

Example

let studentName: string = "Ananya";

Here:

: string

is the type annotation.

Another example:

let studentAge: number = 18;

Scenario

A student registration system may specify:

let studentName: string = "Ravi";

let studentAge: number = 18;

let registrationOpen: boolean = true;

The type annotations clearly indicate what kind of information each variable should contain.

4.2.6 Type Inference

Definition: Type inference is the ability of TypeScript to determine an appropriate type automatically from the value or context of an expression. When a variable is initialized with a value, TypeScript can often infer its type without requiring an explicit type annotation. The inferred type is then used for subsequent type checking. Type inference reduces unnecessary annotations while preserving type safety.

Example

```
let productName = "Laptop";
```

TypeScript infers:

productName → string

Therefore, the following is not compatible:

```
productName = 50000;
```

Another Example

```
let productPrice = 50000;
```

TypeScript infers:

productPrice → number

Scenario

An online shop may write:

```
let stockAvailable = true;
```

TypeScript can infer that stockAvailable is a Boolean value.

4.2.7 Data Types in TypeScript

Definition: A data type specifies the kind of value that a variable, parameter, property, or expression is expected to contain. TypeScript uses data types to check whether values are being used consistently and whether operations are appropriate for those values. Common basic types include string, number, and boolean, while arrays are used to store collections of values. Type information is primarily used during development and is not retained as TypeScript type annotations in the generated JavaScript.

the important types are:

string

number

boolean

array

4.2.8 The string Data Type

Definition: The string data type represents textual information in TypeScript. String values are normally written inside single quotes, double quotes, or template literals. Strings are used to represent information such as names, addresses, product titles, messages, and course names. TypeScript ensures that a variable declared as string receives compatible textual values.

Syntax

```
let variableName: string = "text";
```

Example

```
let studentName: string = "Kavya";
```

Another example:

```
let productName: string = "Laptop";
```

Scenario

An online shopping website stores the product name as:

```
let productName: string =
        "Wireless Mouse";
```

Since the product name is textual information, string is appropriate.

4.2.9 The number Data Type

Definition: The number data type represents numerical values in TypeScript. It is used for both integer and floating-point values under JavaScript's number representation. Numbers can participate in arithmetic operations such as addition, subtraction, multiplication, and division. It is commonly used for prices, quantities, marks, ages, and measurements.

Syntax

```
let variableName: number = numericValue;
```

Example

```
let productPrice: number = 50000;
```

Another example:

```
let studentPercentage: number = 85.5;
```

Scenario

An e-commerce website stores:

```
let productPrice: number = 1500;
let cartQuantity: number = 3;
```

These values can be used in calculations.

```
let totalPrice: number = productPrice * cartQuantity;
```

4.2.10 The boolean Data Type

Definition: The boolean data type represents a logical value that can be either true or false. Boolean values are commonly used to represent conditions, status information, and decisions in a program. They are especially useful when a website needs to determine whether something is available, valid, active, or completed. TypeScript checks that Boolean variables contain only compatible Boolean values.

Syntax

```
let variableName: boolean = true;
```

Example

```
let admissionOpen: boolean = true;
```

Another example:

```
let productAvailable: boolean = false;
```

Scenario

An online shop may store the availability of a product:

```
let productAvailable: boolean = true;
```

The website can later use this value to decide whether the Buy Now option should be available.

4.2.11 Arrays

Definition: An array is a data structure used to store multiple related values in a single variable. In TypeScript, the expected type of the array elements can be specified so that incompatible values can be detected. Array elements are arranged in an ordered collection and can be accessed using indexes. Arrays are useful for storing lists such as student names, product prices, course names, and marks.

Syntax

```
let arrayName: dataType[] = [
        value1,
        value2
];
```

String Array Example

```
let courses: string[] = [
        "HTML",
        "CSS",
        "TypeScript"
];
```

Number Array Example

```
let marks: number[] = [
        80,
        75,
        90
];
```

Accessing an Array Element

```
console.log(courses[0]);
```

Output

HTML

Scenario

A learning website may store the available courses:

```
let courseNames: string[] = [
        "Web Programming",
        "Python",
        "Database Systems"
];
```

All elements are expected to be strings.

4.2.12 Functions in TypeScript

Definition: A function is a reusable block of program statements designed to perform a specific task. In TypeScript, data types can be specified for function parameters and for the value returned by the function. These type annotations allow the compiler to check whether appropriate arguments are supplied and whether the function returns the expected type. Functions help reduce repeated code and organize applications into smaller reusable operations.

Basic Syntax

```
function functionName(
        parameter: dataType
): returnType {
        // Statements
}
```

4.2.13 Function without Parameters

A function does not always need input values.

Example

```
function showWelcome(): void {

        console.log(
                "Welcome to College Portal"
        );
}
```

showWelcome();

Output

Welcome to College Portal

Here:

showWelcome

→ Function name

void

→ Function does not return a value

Scenario

A website may use a function to display a fixed welcome message when required.

4.2.14 Function Parameters

Definition: A parameter is a named variable declared in a function definition to receive a value when the function is called. TypeScript allows each parameter to have an explicit type, which helps ensure that callers supply appropriate arguments. Parameters allow the same function to operate on different input values. They are commonly used for names, quantities, prices, marks, and other information supplied to an operation.

Syntax

```
function functionName(
        parameterName: dataType
) {
}
```

Example

```
function showStudent(
        studentName: string
): void {
        console.log(
                "Student: " + studentName
        );
}
```

```
showStudent("Ananya");
```

Output

```
Student: Ananya
```

Incorrect Call

```
showStudent(100);
```

TypeScript reports an error because the function expects a string.

Scenario

A student portal may pass the student's name to a function responsible for displaying a welcome message.

4.2.15 Multiple Function Parameters

A function may receive more than one input.

Example

```
function showProduct(
        productName: string,
        productPrice: number
): void {

        console.log(
                productName +
                " - ₹" +
                productPrice
        );
}

showProduct(
        "Laptop",
        50000
);
```

Output

Laptop – ₹50000

Scenario

An online store can send both the product name and price to the same function for display.

4.2.16 Function Return Type

Definition: A return type specifies the kind of value that a TypeScript function is expected to return to its caller. It is written after the parameter list using a colon followed by the type name. TypeScript can verify whether the value returned by the function is compatible with the declared return type. Explicit return types can make the purpose and expected result of a function clearer.

Syntax

```
function functionName(
        parameter: dataType
): returnType {

        return value;
}
```

Example

```
function calculateTotal(
        price: number,
        quantity: number
): number {

        return price * quantity;
}
```

Using the function:

```
let totalAmount: number =
        calculateTotal(500, 3);
```

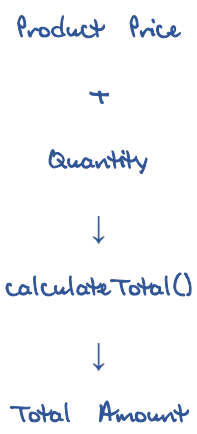
```
console.log(totalAmount);
```

Output

1500

Scenario

A shopping website uses a function to calculate the total price from the product price and quantity.



4.2.17 The void Return Type

Definition: The void type is commonly used to indicate that a function does not return a useful value to its caller. Such a function may still perform an action such as displaying information, updating content, or writing to the console. Declaring a function with void makes it clear that the caller should not expect a returned result. It is frequently used for functions whose purpose is to perform an operation rather than calculate a value.

Example

```
function displayMessage(
        message: string
): void {

        console.log(message);
}

displayMessage(
        "Registration Successful"
);
```

Output

Registration Successful

Scenario

A registration website may use a function only to display a confirmation message. Since no value needs to be returned, void is appropriate.

4.2.18 Basic Function Working Principle

The function process can be represented as:

Function Call

↓

Arguments Supplied

↓

Parameters Receive Values

↓

Function Executes

↓

Return Result

For example:

calculateTotal(500, 2);

Working:

500 → price

2 → quantity

↓

500 + 2

↓

1000

↓

Returned to Caller

4.2.19 Scenario - Shopping Cart

Consider an online shopping website.

The website needs to store:

Product Name

Product Price

Quantity

Availability

TypeScript variables can represent them as:

```
const productName: string =
        "Wireless Keyboard";


let productPrice: number = 1500;
let cartQuantity: number = 2;
let productAvailable: boolean = true;
```

A function can calculate the total amount:

```
function calculateTotal(
        price: number,
        quantity: number
): number {
        return price * quantity;
}
```

The relationship is:

Variables

↓

Store Product Data

↓

Function

↓

Process Product Data

↓

Return Total

4.2.2D Program

The following program demonstrates variables, basic data types, arrays, and functions in TypeScript.

```
const websiteName: string ="ABC Online Store";

let productName: string = "Wireless Mouse";

let productPrice: number =800;

let cartQuantity: number =2;

let productAvailable: boolean =true;

let categories: string[] = [

        "Electronics",

        "Accessories",

        "Computers"

];


function calculateTotal(

        price: number,

        quantity: number

): number {

        return price * quantity;

}

function displayProduct(

        name: string,

        total: number

): void {

        console.log(

                "Product: " + name

        );

        console.log(

                "Total Price: ₹" + total

        );
```

```
}
let totalAmount: number =
        calculateTotal(
                productPrice,
                cartQuantity
        );
console.log("Website: " + websiteName);
console.log( "Available: " +productAvailable);
console.log("Category: " +categories[1]);
displayProduct(productName,totalAmount);
```

4.2.21 Program Output

```
Website: ABC Online Store
Available: true
Category: Accessories
Product: Wireless Mouse
Total Price: ₹1600
```

4.2.22 Program Explanation

The statement:

```
const websiteName: string = "ABC Online Store";
```

stores the website name as a string.

The variables:

```
let productPrice: number = 800;
let cartQuantity: number = 2;
```

store numerical values.

The variable:

```
let productAvailable: boolean =    true;
```

stores a Boolean status.

The array:

```
let categories: string[] = [
        "Electronics",
        "Accessories",
        "Computers"
];
```

stores multiple string values.

The function:

```
function calculateTotal(
        price: number,
        quantity: number
): number
```

accepts two numerical parameters and returns a numerical result.

The calculation:

price * quantity

produces:

800 * 2 = 1600

The function:

displayProduct()

receives a product name and total amount and displays them. Its return type is void because it does not return a value.

4.2.23 Variables and Functions - Overall Relationship

Variables

↓

Store Data

↓

Data Types

↓

Describe Expected Values

↓

Functions

↓

Receive Typed Parameters

↓

Process Data

↓

Return Typed Result

4.2.24 Important Points

let is used when a variable needs to be reassigned.

const is preferred when reassignment is not required.

A type annotation is written using:

: type

Examples:

: string

: number

: boolean

TypeScript can also infer many types automatically.

Functions can have typed parameters:

```
function showName(
        name: string
)
```

and typed return values:

```
function calculate(): number
```

A function that does not return a useful value can use:

:void

A simple memory sequence is:

Variable → Stores Value

Type → Describes Value

Function → Processes Value

4.2.25 Scenario-Based Questions

Question 1

A college website needs to store the name of a student. Which TypeScript data type is appropriate?

Answer: string.

```
let studentName: string =
    "Ananya";
```

Question 2

An online store needs to store the price of a laptop. Which data type should be used?

Answer: number.

```
let productPrice: number =
    50000;
```

Question 3

A website stores whether admission registration is currently open. Which data type is suitable?

Answer: boolean.

```
let registrationOpen: boolean =
    true;
```

Question 4

A shopping website needs to store several product categories in one variable. Which structure can be used?

Answer: An array.

```
let categories: string[] = [
    "Mobile",
    "Laptop",
    "Accessories"
];
```

Question 5

A variable representing available seats changes whenever a student completes admission. Should let or const be used?

Answer: let, because the value needs to be reassigned.

Question 6

A function receives product price and quantity and returns their product. Which return type is appropriate?

Answer: number.

```
function calculateTotal(
    price: number,
    quantity: number
): number {
    return price * quantity;
}
```

Question 7

A function only displays a welcome message and does not return a value. Which return type can be used?

Answer: void.

4.2.26 Important Questions

1. What is a variable in TypeScript?

A variable is a named storage location used to hold program data. TypeScript allows the expected type of the stored value to be specified or inferred.

2. What is the difference between let and const?

let allows the variable to be reassigned after declaration. const does not allow the variable binding to be reassigned after initialization.

3. What is a type annotation?

A type annotation explicitly specifies the expected type of a variable or other program element. It is written using a colon followed by the type name.

4. What is type inference?

Type inference is TypeScript's ability to determine an appropriate type automatically from the value or context of an expression.

5. Name three basic TypeScript data types.

Three commonly used basic types are string, number, and boolean.

6. What is an array?

An array is an ordered collection used to store multiple related values. TypeScript can specify the expected type of its elements.

7. What is a function?

A function is a reusable block of code that performs a particular operation. TypeScript allows parameter types and return types to be specified.

8. What is a function parameter?

A parameter is a named variable in a function declaration that receives an argument supplied when the function is called.

9. What is a return type?

A return type specifies the kind of value that a function is expected to return.

10. What does void indicate?

void commonly indicates that a function does not return a useful value to its caller.

4.2.27 Summary

Variables store information used by a TypeScript program. Modern TypeScript programs commonly use let for values that need reassignment and const when reassignment is not required.

TypeScript provides type annotations and type inference to describe and check data. Common beginner data types include string, number, boolean, and arrays.

Functions organize reusable operations. TypeScript allows function parameters and return values to have explicit types, helping the compiler detect incorrect function calls and results.

The fundamental relationship is:

Variable

↓

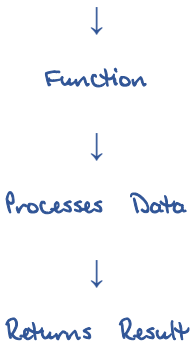
Stores Data

↓

Data Type

↓

Specifies Expected Data


Function
Processes Data
Returns Result

4.3 Optional, Default and Rest Parameters

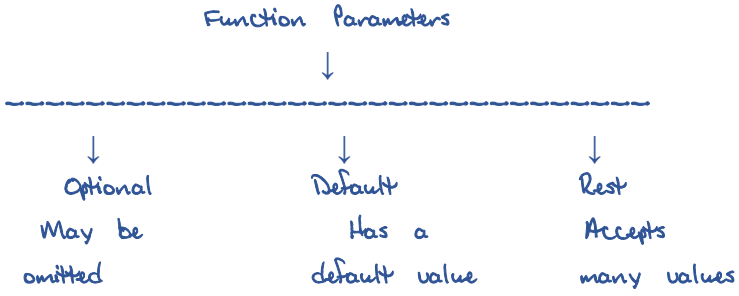
4.3.1 Introduction

Functions often need input values called parameters. In some situations, every parameter must be supplied whenever the function is called. In other situations, some information may be optional, some parameters may have a commonly used value, or the number of input values may not be known in advance.

TypeScript provides different parameter mechanisms to handle these situations. The important forms covered in this topic are optional parameters, default parameters, and rest parameters.

For example, an online shopping application may have a customer name that is compulsory, a coupon code that is optional, a quantity that normally defaults to one, and a function that receives several product prices.

The basic relationship is:



4.3.2 Function Parameters

Definition: A parameter is a named variable declared in a function definition that receives a value when the function is called. Parameters allow a function to work with different input values instead of using fixed values. TypeScript allows each parameter to have an expected data type. Values supplied to parameters when calling a function are called arguments.

Example

```
function showProduct(
        name: string,
        price: number
): void {
        console.log(name);
        console.log(price);
}
showProduct(
        "Laptop",
```

50000

);

Here:

name, price

→ Parameters

"Laptop", 50000

→ Arguments

Scenario

An e-commerce website can pass a product name and price to the same function for displaying different products.

4.3.3 Need for Special Parameter Types

Consider a function used for customer registration.

```
function registerCustomer(
    name: string,
    phone: string
): void {
    console.log(name);
    console.log(phone);
}
```

This function requires both values.

However, suppose the phone number is not compulsory. The programmer needs a way to indicate that the parameter may be omitted.

Similarly, an online shopping function may normally use a quantity of 1 unless the customer chooses another quantity.

Another function may need to receive several product prices without knowing in advance how many products will be added.

TypeScript provides:

Optional Parameter

→ Value may be omitted

Default Parameter

→ Uses a predefined value when omitted

Rest Parameter

→ Accepts multiple values

4.3.4 Optional Parameters

Definition: An optional parameter is a function parameter that does not have to be supplied when the function is called. In TypeScript, an optional parameter is written using the ? symbol after the parameter name. If the argument is omitted, the parameter may have the value undefined inside the function. Optional parameters are useful when certain information is not required in every function call.

Syntax

```
function functionName(
    requiredParameter: type,
    optionalParameter?: type
) {
}
```

Example

```
function showStudent(
    name: string,
    department?: string
): void {

    console.log(
        "Student: " + name
    );

    if (department !== undefined) {

        console.log(
            "Department: " + department
```

```
        );
    }
}
```

Function call with both values:

```
showStudent(
    "Ananya",
    "CSE"
);
```

Output

Student: Ananya

Department: CSE

Function call without the optional value:

```
showStudent("Ananya");
```

Output

Student: Ananya

Here:

department?: string

means that department may be supplied or omitted.

4.3.5 Optional Parameter Symbol ?

The question mark: ?

indicates that a parameter is optional.

Example:

phoneNumber ?: string

This means: phoneNumber

→ May contain a string

or

→ May be omitted

For optional parameters are normally placed after required parameters.

Correct:

```
function register(
        name: string,
        phone?: string
) {
}
```

Scenario

A registration website requires the student's name but does not require an alternate phone number.

```
function registerStudent(
        studentName: string,
        alternatePhone?: string
): void {
        console.log(studentName);
}
```

The registration can therefore proceed even if the alternate phone number is not provided.

4.3.16 Scenario - Optional Coupon Code

An online shopping website may allow a customer to enter a coupon code, but a coupon is not compulsory.

```
function placeOrder(
        productName: string,
        couponCode?: string
): void {
        console.log( "Product: " + productName );
        if (couponCode !== undefined) {
                console.log(
                        "Coupon: " + couponCode
                );
        }
}
```

Without coupon:

```
placeOrder("Laptop");
```

Output

```
Product: Laptop
```

With coupon:

```
placeOrder(
        "Laptop",
        "SAVE10"
);
```

Output

```
Product: Laptop
Coupon: SAVE10
```

The coupon is therefore optional.

4.3.7 Default Parameters

Definition: A default parameter is a function parameter that has a predefined value that is used when the caller does not provide an argument for that parameter. The default value is written directly in the function declaration using the assignment operator. Default parameters reduce the need to repeatedly supply a commonly used value. They are useful when a function normally operates with a standard value but still allows the caller to provide a different one.

Syntax

```
function functionName(
        parameter: type = defaultValue
) {
}
```

Example

```
function calculatePrice(
        price: number,
        quantity: number = 1
): number {
```

```
    return price * quantity;
}
```

Call without quantity:

```
console.log(
    calculatePrice(500)
);
```

Output

500

Because:

quantity

→ Not supplied

→ Default value = 1

4.3.8 Changing a Default Parameter Value

A caller can still supply another value.

```
console.log(
    calculatePrice(
        500,
        3
    )
);
```

Output

1500

In this case:

Default quantity = 1

but

Caller supplied = 3

Therefore quantity = 3

Scenario

When a customer opens a product page, the initial purchase quantity may be one.

```
function calculateOrder(
        productPrice: number,
        quantity: number = 1
): number {

        return productPrice * quantity;
}
```

If the customer does not change the quantity, the website uses one item.

4.3.9 Default Parameter and undefined

A default value is also used when undefined is supplied for that parameter.

Example

```
function greetUser(
        name: string = "Guest"
): void {

        console.log(
                "Welcome " + name
        );
}
```

Call:

```
greetUser();
```

Output

Welcome Guest

Call:

```
greetUser("Ravi");
```

Output

Welcome Ravi

Scenario

A website can display:

Welcome Guest

when a visitor's name is not supplied, while displaying the actual name for a registered user.

4.3.10 Optional Parameter vs Default Parameter

Students should clearly understand the difference.

Optional Parameter	Default Parameter
Argument may be omitted	Argument may be omitted
Written using ?	Written using =
May become undefined	Receives the specified default value
No automatic value is provided	Automatic default value is provided

Optional

```
function showUser( name?: string) { }
```

Default

```
function showUser( name: string = "Guest") { }
```

Simple:

Optional

→ Value may be absent

Default

→ Value is automatically supplied

when needed

4.3.11 Rest Parameters

Definition: A rest parameter allows a function to receive zero or more additional arguments and collect them into a single array. In TypeScript, a rest parameter is written using three dots ... before the parameter name and is normally given an array type. It is useful when the number of values supplied to a function cannot be determined in advance. A rest parameter must be the final parameter in the parameter list.

Syntax

```
function functionName( ...parameterName: type[]) { }
```

Example

```
function showPrices(...prices: number[]): void {
        console.log(prices);
}
```

Calling:

```
showPrices(
        100,
        200,
        300
);
```

The rest parameter collects:

100

200

300

into:

[100, 200, 300]

4.3.12 Three Dots ...

The syntax: ... is used before a rest parameter.

Example:

...prices: number[]

It means:

Receive remaining arguments

↓

Collect them

↓

Store them in an array

Example

```
function displayMarks( ...marks: number[]
): void {
        console.log(marks);
}
displayMarks(
        80,
        75,
        90,
        85
);
```

The marks parameter contains:

[80, 75, 90, 85]

Scenario

A result application may receive marks for several subjects. Instead of creating separate parameters for every subject, a rest parameter can collect all marks.

4.3.13 Rest Parameter with a Loop

Because a rest parameter behaves as an array, its values can be processed using a loop.

Example

```
function calculateTotal( ...prices: number[]
): number {
        let total = 0;
        for (const price of prices) {
                total += price;
        }
        return total;
}
```

Function call:

console.log(calculateTotal(500,300,200));

Output

1000

Working

Arguments:

500, 300, 200

↓

Rest Parameter

↓

[500, 300, 200]

↓

Loop

↓

500 + 300 + 200

↓

1000

4.3.14 Shopping Cart

A customer may add a different number of products to a shopping cart.

One customer may purchase:

Laptop

Mouse

Another may purchase:

Laptop

Mouse

Keyboard

Headphone

The number of prices is therefore not fixed.

A rest parameter can accept all prices.

function calculateCartTotal(

```
    ...productPrices: number[]
): number {
    let total = 0;
    for (
        const price
        of productPrices
    ) {
        total += price;
    }
    return total;
}
```

Call:

```
console.log(
    calculateCartTotal(
        50000,
        800,
        1500
    )
);
```

Output

52300

4.3.15 Required, Optional, Default and Rest Parameters

The complete comparison is:

Parameter Type	Purpose	Example
Required	Must normally be supplied	name: string
Optional	May be omitted	phone?: string
Default	Uses predefined value	quantity: number = 1
Rest	Receives multiple values	...prices: number[]

Simple Representation

Required

→ Must supply

Optional

→ May supply

Default

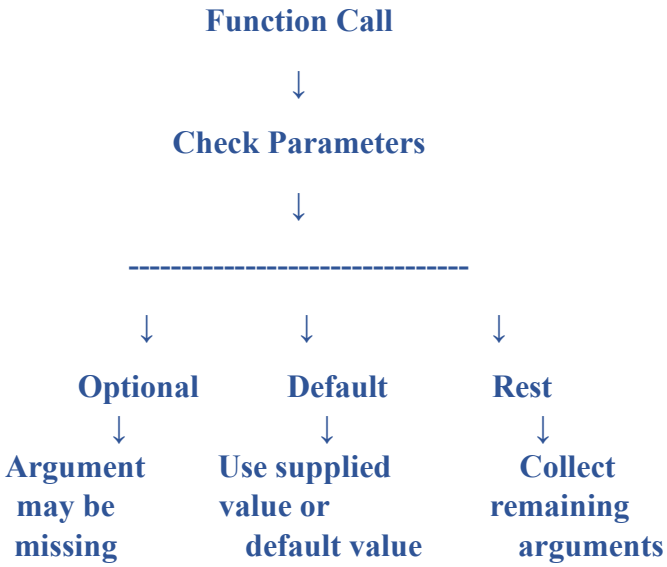
→ Value already available

Rest

→ Supply many values

4.3.16 Working Principle

The three parameter forms can be represented as:



4.3.17 Scenario - Online Order

Consider an online shopping application.

The application needs:

a product name,

an optional coupon code,

a default quantity of one, and the ability to calculate several product amounts.

These requirements can be handled using TypeScript parameter features.

4.3.18 Program

```
function displayOrder(
        productName: string,
        couponCode?: string
): void {
        console.log("Product: " +productName);
        if (
                couponCode !== undefined
        ) {
                console.log("Coupon: " + couponCode);
        }
}
function calculateProductTotal(
        price: number,
        quantity: number = 1
): number {
        return price * quantity;
}
function calculateCartTotal(
        ...amounts: number[]
): number {
        let total = 0;

        for (
                const amount
                of amounts
        ) {
                total += amount;
        }
        return total;
}
```

```
displayOrder(
        "Wireless Mouse",
        "JAVED"
);
const mouseTotal =
        calculateProductTotal(
                800,
                2
        );
const keyboardTotal =
        calculateProductTotal(
                1500
        );
const cartTotal =
        calculateCartTotal(
                mouseTotal,
                keyboardTotal
        );
console.log(
        "Mouse Total: " + ₹
        mouseTotal
);
console.log(
        "Keyboard Total: " + ₹
        keyboardTotal
);
console.log(
        "Cart Total: " + ₹
        cartTotal
);
```

4.3.19 Program Output

Product: Wireless Mouse

Coupon: SAVE10

Mouse Total: 1600 ₹

Keyboard Total: 1500 ₹

Cart Total: 3100 ₹

4.3.20 Program Explanation

The function:

displayOrder()

contains:

couponCode?: string

Therefore, the coupon code is optional.

The function:

calculateProductTotal()

contains:

quantity: number = 1

Therefore, if quantity is not supplied, its default value is 1.

For:

calculateProductTotal(

1500

);

the calculation becomes:

1500 * 1

=

1500

The function:

calculateCartTotal()

contains:

...amounts: number[]

Therefore, it can receive several numerical values.

The call:

```
calculateCartTotal(
        mouseTotal,
        keyboardTotal
);
```

collects the values into an array and calculates their sum.

Thus, the program demonstrates:

Optional

→ couponCode

Default

→ quantity = 1

Rest

→ ...amounts

4.3.21 Important Points

An optional parameter uses:

?

Example:

department?: string

A default parameter uses:

=

Example:

quantity: number = 1

A rest parameter uses:

...

Example:

...prices: number[]

A rest parameter should appear last in the parameter list.

The simplest memory rule is:

? → Optional

= → Default

... → Rest

4.3.22 Scenario-Based Questions

Question 1

A college registration form requires a student name but allows the alternate phone number to be omitted. Which parameter type should be used for the alternate phone number?

Answer: An optional parameter.

alternatePhone?: string

Question 2

A shopping website assumes that a customer purchases one item unless another quantity is supplied. Which parameter type is appropriate?

Answer: A default parameter.

quantity: number = 1

Question 3

A function needs to receive an unknown number of product prices. Which parameter should be used?

Answer: A rest parameter.

...prices: number[]

Question 4

Which symbol identifies an optional parameter?

Answer:

?

Question 5

Which symbol is used before a rest parameter?

Answer:

...

Question 6

What happens if the caller does not provide a value for a default parameter?

Answer: The predefined default value is used.

Question 7

A function has:

```
function greet(
        name: string = "Guest"
)
```

What value is used when greet() is called without an argument?

Answer: "Guest".

Question 8

What does a rest parameter collect its arguments into?

Answer: An array.

4.3.23 Important Questions

1. What is an optional parameter?

An optional parameter is a function parameter that may be omitted when the function is called. It is written using the ? symbol after the parameter name.

2. Give the syntax of an optional parameter.

parameterName?: type

3. What is a default parameter?

A default parameter has a predefined value that is used when the caller does not supply an argument for that parameter.

4. Give the syntax of a default parameter.

parameterName: type = defaultValue

5. What is a rest parameter?

A rest parameter accepts zero or more remaining arguments and collects them into an array.

6. Give the syntax of a rest parameter.

...parameterName: type[ ]

7. What is the difference between optional and default parameters?

An optional parameter may be absent and can therefore be undefined. A default parameter automatically receives a predefined value when an argument is not supplied.

8. Why are rest parameters useful?

Rest parameters are useful when a function needs to accept a variable number of input values.

9. Where should a rest parameter appear?

A rest parameter must appear as the final parameter in the function parameter list.

4.3.24 Summary

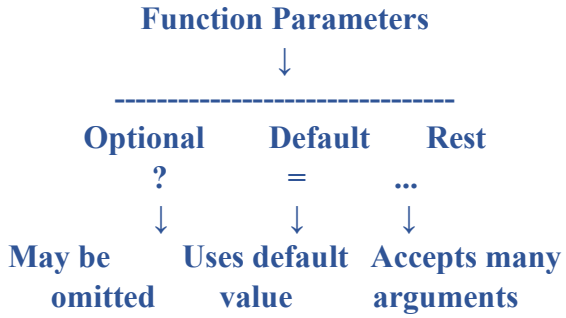
TypeScript provides optional, default, and rest parameters to make functions more flexible.

An optional parameter is identified using ? and allows an argument to be omitted.

A default parameter contains a predefined value that is used when the caller does not provide a value.

A rest parameter uses ... to collect multiple remaining arguments into an array.

The fundamental relationship is:



The simplest is:

Optional

→ Argument may be omitted

Default

→ Uses a predefined value

Rest

→ Accepts multiple arguments

4.4 Arrow Functions

4.4.1 Introduction

Functions are used to organize reusable operations in TypeScript programs. In the previous topics, functions were written using the traditional function keyword with parameters and return types.

TypeScript also supports arrow functions, which provide a shorter way of writing function expressions. Arrow functions are commonly used for small calculations, callbacks, array operations, and other short operations in web applications.

For example, a shopping website may need a small function to calculate the total price of a product.

Traditional function:

```
function calculateTotal(
        price: number,
        quantity: number
): number {
        return price * quantity;
}
```

The same operation can be written using an arrow function.

```
const calculateTotal = (
        price: number,
        quantity: number
): number => {
        return price * quantity;
};
```

The important symbol is:

=> It is called the arrow.

4.4.2 Arrow Function

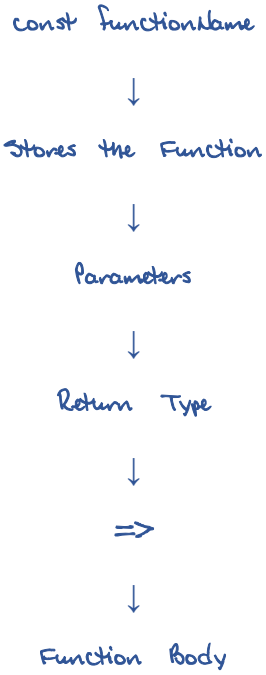
Definition: An arrow function is a compact form of function expression written using the arrow symbol =>. It can receive parameters, use TypeScript type annotations, perform operations, and return values.

Arrow functions are particularly convenient for short functions and callback operations.

Basic Syntax

```
const functionName = (
        parameter: type
): returnType => {
        // statements
};
```

Basic Structure



4.4.3 Need for Arrow Functions

Traditional functions are completely valid in TypeScript.

Example:

```
function add(
        number1: number,
        number2: number
): number {
        return number1 + number2;
}
```

But the same function can be written more compactly:

```
const add = (
        number1: number,
```

```
    number2: number
): number => {
    return number1 + number2;
};
```

For short operations:

Traditional Function

↓

Usually Longer Syntax

Arrow Function

↓

More Compact Syntax

Arrow functions are useful when a web application contains many short operations such as:

Calculate Price

Display Message

Check Stock

Calculate Discount

Process Array Values

Execute Callback

4.4.4 Traditional Function and Arrow Function

Traditional Function

```
function calculateTotal(
    price: number,
    quantity: number
): number {
    return price * quantity;
}
```

Arrow Function

```
const calculateTotal = (
```

```
        price: number,
        quantity: number
): number => {
        return price * quantity;
};
```

Both can produce the same result.

```
console.log(
        calculateTotal(500, 2)
);
```

Output:

1000

Simple memory:

function

→ Traditional Function

=>

→ Arrow Function

4.4.5 Arrow Function without Parameters

An arrow function does not always require input parameters.

Example:

```
const showWelcome = (): void => {
        console.log(
                "Welcome to College Portal"
        );
};
```

Function call:

```
showWelcome();
```

Output:

Welcome to College Portal

Here:

()

↓

No Parameters

void

↓

No Value Sent Back

=>

↓

Arrow Function

Scenario

A college website may display a fixed welcome message when the page starts.

4.4.16 Arrow Function with One Parameter

An arrow function can receive one parameter.

```
const showProduct = (
        productName: string
): void => {
        console.log(
                "Product: " + productName
        );
};
```

Function call:

```
showProduct("Laptop");
```

Output:

```
Product: Laptop
```

Working:

"Laptop"

↓

productName

↓

Arrow Function

↓

Display Product

Scenario

An e-commerce website can pass the selected product name to an arrow function for display.

4.4.7 Arrow Function with Multiple Parameters

An arrow function can receive multiple parameters.

```
const calculateTotal = (
        productPrice: number,
        quantity: number
): number => {
        return productPrice * quantity;
};
```

Function call:

```
console.log(
        calculateTotal(
                800,
                2
        )
);
```

Output:

1600

Working:

8000 → productPrice

2 → quantity

↓

calculateTotal()

↓

8000 * 2

↓

16000

↓

Return Result

Scenario

A shopping cart requires both product price and quantity to calculate the total amount.

4.4.8 Type Annotations in Arrow Functions

Type annotations specify the type of data expected by an arrow function.

Example:

```
const calculateDiscount = (
        price: number
): number => {
        return price * 0.10;
};
```

Here:

price: number

means that price must contain a numerical value.

And:

: number =>

means that the function returns a number.

Incorrect call:

```
calculateDiscount(
        "Five Thousand"
```

);

TypeScript reports an error because the function expects a number.

4.4.9 Return Type in Arrow Functions

The return type tells TypeScript what type of value should be sent back by the function.

Syntax

```
const functionName = (
    parameter: type
): returnType => {
    return value;
};
```

Example:

```
const getStudentResult = (
    mark: number
): string => {
    if (mark >= 50) {
        return "PASS";
    }
    return "FAIL";
};
```

Function call:

```
console.log(
    getStudentResult(75)
);
```

Output:

```
PASS
```

Working:

mark = 75

↓

75 >= 50

↓

true

↓

"PASS"

↓

return

Scenario

A student-result portal can use an arrow function to convert numerical marks into "PASS" or "FAIL".

4.4.10 Arrow Function with void

When an arrow function performs an action but does not send any value back, its return type can be written as void.

Example:

```
const displayMessage = (
    message: string
): void => {
    console.log(message);
};
```

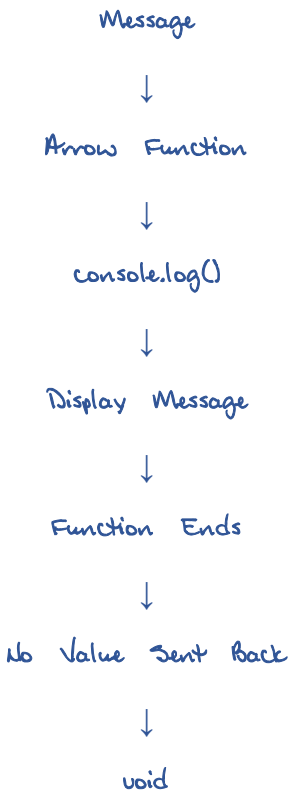
Function call:

```
displayMessage(
    "Registration Successful"
);
```

Output:

Registration Successful

Working:



Remember:

console.log()

→ Displays something

return

→ Sends a value back

void

→ No value sent back

This keeps the void explanation consistent with the function material you have already taught.

4.4.11 Explicit Return

When curly braces { } are used and a result must be returned, the return keyword is written explicitly.

Example:

```
const calculateSquare = (
        value: number
): number => {
```

```
    return value * value;
};
```

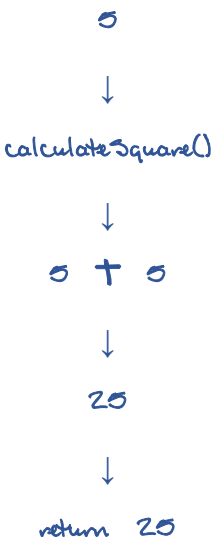
Function call:

```
console.log(
    calculateSquare(5)
);
```

Output:

25

Working:



This is called an explicit return because return is directly written.

4.4.12 Implicit Return

An arrow function containing only a single expression can return the expression result without explicitly writing return.

Normal arrow function:

```
const calculateTotal = (
    price: number,
    quantity: number
): number => {
```

```
    return price * quantity;
};
```

Short form:

```
const calculateTotal = (
    price: number,
    quantity: number
): number =>
    price * quantity;
```

Function call:

```
console.log(
    calculateTotal(
        500,
        3
    )
);
```

Output:

1500

Working:

Single Expression

↓

Curly Braces Omitted

↓

return Omitted

↓

Expression Result

Automatically Returned

This behavior is also covered in your uploaded 4.4 material.

Easy Rule

With { }

→ Write return when a value must be returned

Single expression without { }

→ return can be omitted

4.4.13 Arrow Function Returning Boolean

An arrow function can also return a Boolean value.

Example:

```
const checkAvailability = (
        stock: number
): boolean =>
        stock > 0;
```

Function call:

```
console.log(
        checkAvailability(10)
);
```

Output:

true

If:

checkAvailability(0);

Result: false

working:

Stock

↓

stock > 0

↓

true / false

↓

Product Available

or

Product Not Available

4.4.14 Arrow Functions as Callbacks

Arrow functions are commonly used as callback functions.

Example:

```
setTimeout(
    (): void => {
        console.log(
            "Data loaded"
        );
    },
    2000
);
```

Working:

Program Starts

↓

Wait Approximately 2 Seconds

↓

Arrow Callback Executes

↓

"Data loaded"

Output:

Data loaded

Scenario

A webpage may wait for an operation and then display information.

Loading...

↓

Wait

↓

Callback Executes

↓

Data Loaded

This connects arrow functions with asynchronous programming studied earlier.

4.4.15 Arrow Functions with Arrays

Arrow functions are frequently used when processing array elements.

Example:

```
const prices: number[] = [
    500,
    800,
    1500
];
```

Suppose the website provides a 10% discount.

```
const discountedPrices =
    prices.map(
        (price: number): number =>
            price * 0.9
    );
```

Working:

500

800

1500

↓

Each Array Element

↓

Arrow Function

↓

price * 0.9

↓

Discounted Values

remember:

Array Element

↓

Arrow Function

↓

Processed Value

4.4.16 Lexical this

Arrow functions handle this differently from traditional functions.

Traditional Function

→ this can depend on how the function is called

Arrow Function

→ Uses this from the surrounding scope

4.4.17 Traditional Function vs Arrow Function

Traditional Function	Arrow Function
Uses function keyword	Uses =>
Usually longer syntax	Usually more concise
Can use typed parameters	Can use typed parameters
Can specify return type	Can specify return type
Normal this behavior	Lexical this
Useful for general functions	Common for short functions and callbacks

Example:

Traditional

```
function add(
        number1: number,
        number2: number
): number {
        return number1 + number2;
}
```

Arrow

```
const add = (
        number1: number,
        number2: number
): number =>
        number1 + number2;
```

4.4.18 Working Principle of an Arrow Function

Arrow Function Defined

↓

Parameters Specified

↓

Function Called

↓

Arguments Passed

↓

Statements / Expression Executed

↓

Result Returned

or

Action Performed

Example:

```
const calculateTotal = (
    price: number,
    quantity: number
): number =>
    price * quantity;
```

Call:

```
calculateTotal(
    1000,
    2
);
```

Working:

1000 → price

2 → quantity

↓

1000 * 2

↓

2000

↓

Returned

4.4.19 Real Website Scenario - Shopping Cart

Consider an online shopping website that needs to:

Display Product

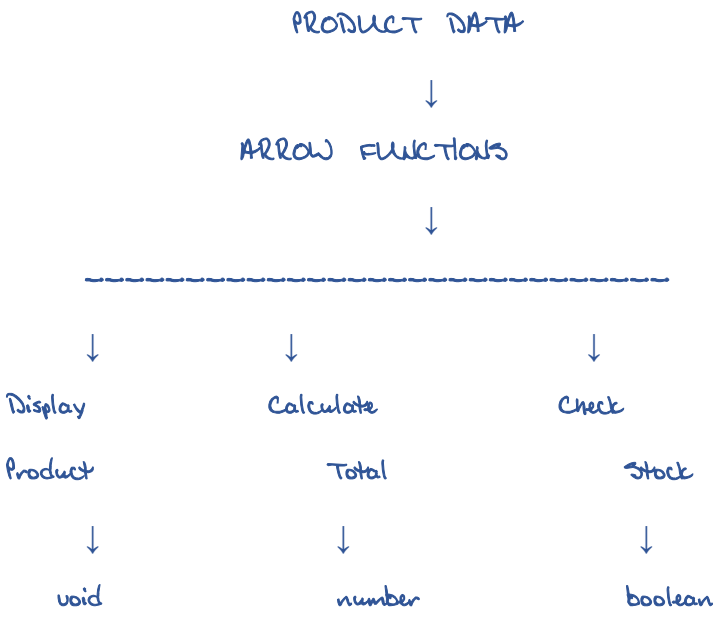
Calculate Total

Check Availability

Arrow functions can perform these small operations.

```
const displayProduct = (
        productName: string
): void => {
        console.log(
                "Product: " + productName
        );
};
const calculateTotal = (
        price: number,
        quantity: number
): number =>
        price * quantity;
const checkAvailability = (
        stock: number
): boolean =>
        stock > 0;
```

Relationship:



4.4.2D Program

```
const websiteName: string =
        "ABC Online Store";
const productName: string =
        "Wireless Mouse";
const productPrice: number =
        800;
const cartQuantity: number =
        2;
const availableStock: number =
        10;
const displayWelcome = (): void => {
        console.log(
                "Welcome to " + websiteName
        );
};
const displayProduct = (
        name: string
): void => {
```

```
        console.log(
            "Product: " + name
        );
};
const calculateTotal = (
        price: number,
        quantity: number
): number =>
        price * quantity;
const checkAvailability = (
        stock: number
): boolean =>
        stock > 0;
displayWelcome();
displayProduct(
        productName
);
const totalAmount: number =
        calculateTotal(
            productPrice,
            cartQuantity
        );
const available: boolean =
        checkAvailability(
            availableStock
        );
console.log( "Total Price: ₹" + totalAmount);
console.log("Available: " + available
);
```

Output

Welcome to ABC Online Store

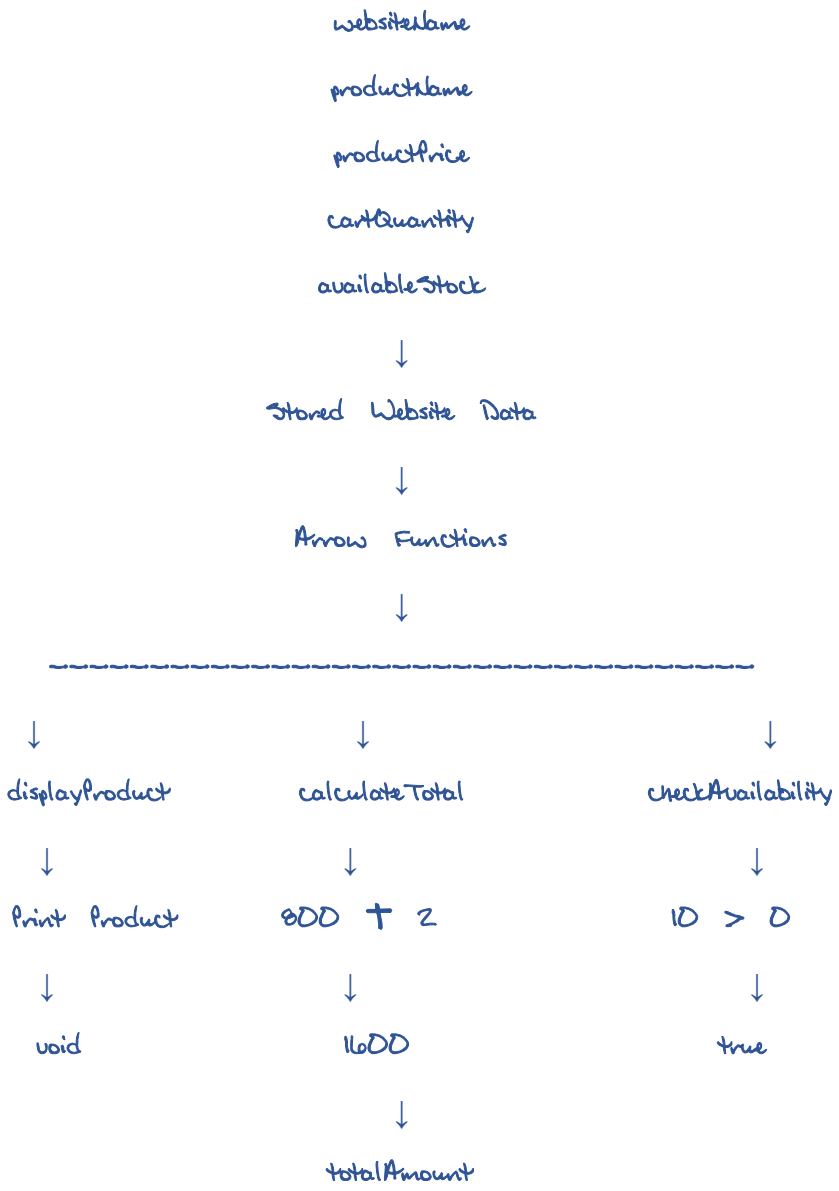
Product: Wireless Mouse

Total Price: ₹1600

Available: true

This follows the shopping-cart example in your uploaded notes, while keeping the output syntax valid and easy for beginners to read.

4.4.21 Program Working



4.4.22 Important Points

1. Arrow functions use the symbol =>.
2. They are another way of writing functions.
3. They can receive zero, one, or multiple parameters.
4. TypeScript data types can be specified for parameters.
5. A return type can also be specified.
6. void means no value is sent back.
7. return explicitly sends a result back.
8. A single-expression arrow function can use implicit return.
9. Arrow functions are commonly used for callbacks and array operations.
10. Arrow functions use this from their surrounding scope.

4.4.23 Important Questions

1. What is an arrow function?
An arrow function is a compact form of function expression that uses the => symbol.

2. Give the basic syntax.

```
const functionName = (
    parameter: type
): returnType => {
    // statements
};
```

3. Which symbol identifies an arrow function?

=>

4. Can an arrow function receive multiple parameters?
Yes.

5. Can it return a value?
Yes. It can use explicit or implicit return.

6. What is implicit return?
When a single-expression arrow function returns its expression result without explicitly writing return.

7. What does void mean?
The function performs its work but does not send a value back.

8. Where are arrow functions commonly used?

Short calculations, callbacks, array processing, and other concise function expressions.

4.4.24 Summary

An arrow function is a concise way to write a function expression in TypeScript.

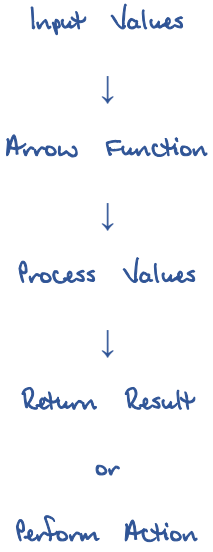
Basic form:

```
const functionName = (
    parameter: type
): returnType => {
    // statements
};
```

For a simple single-expression calculation:

```
const calculateTotal = (
    price: number,
    quantity: number
): number =>
    price * quantity;
```

The fundamental idea is:



4.5 Interfaces and Duck Typing

4.5.1 Introduction

Web applications frequently work with objects that contain related information. For example, an online shopping website may represent a product using properties such as product name, price, and availability.

```
const product = {
    name: "Laptop",
    price: 50000,
    available: true
};
```

When many parts of an application use similar objects, it is useful to clearly specify what properties those objects are expected to contain and what type of values those properties should hold.

TypeScript provides interfaces for describing such object structures.

Another important TypeScript idea is structural typing, often explained informally using the term duck typing. It means that TypeScript mainly checks whether an object has the required structure rather than requiring the object to be created from a particular named type.

4.5.2 Interface

Definition: An interface in TypeScript is a type declaration that describes the expected structure of an object. It can specify the names and types of properties and methods that an object should contain. Interfaces allow different parts of a program to use a common structure while TypeScript checks whether supplied objects satisfy that structure. Interfaces are primarily used during development and type checking and do not create JavaScript objects at runtime.

Basic Syntax

```
interface InterfaceName {
    propertyName: type;
}
```

Example

```
interface Product {
    name: string;
    price: number;
```

```
}
```

This interface describes an object that should contain:

name → string

price → number

Scenario

An e-commerce website may require every product to contain:

Product Name

Product Price

The Product interface can define that common structure.

4.5.3 Need for Interfaces

Consider several product objects:

```
const laptop = {
        name: "Laptop",
        price: 50000
};
const mouse = {
        name: "Wireless Mouse",
        price: 800
};
```

Both objects follow the same structure.

Without a common type description, the expected structure is only understood by reading the code.

An interface makes the requirement explicit:

```
interface Product {
        name: string;
        price: number;
}
```

Now objects can be checked against this structure.

```
const laptop: Product = {
```

```
    name: "Laptop",
    price: 50000
};
```

Website Object

↓

Needs a Common Structure

↓

Interface

↓

Defines Required Properties

↓

TypeScript Checks Object

Scenario

A product-listing page may receive hundreds of products. An interface helps ensure that each product contains the required properties before the program uses them.

4.5.4 Declaring an Interface

Syntax

```
interface InterfaceName {
    property1: type;
    property2: type;
}
```

Example

```
interface Student {
    name: string;
    rollNumber: number;
}
```

Here:

Student

→ Interface name

name

→ String property

rollNumber

→ Number property

The interface describes a structure but does not itself create a student object.

4.5.5 Creating an Object Using an Interface

Example

```
interface Product {
        name: string;
        price: number;
}
const laptop: Product = {
        name: "Laptop",
        price: 50000
};
```

The object:

laptop

satisfies the Product interface because it contains both required properties with compatible types.

Scenario

A shopping website can create several products using the same interface:

```
const mobile: Product = {
        name: "Mobile",
        price: 20000
};
```

The interface gives both objects a common expected structure.

4.5.6 Required Properties

Definition: A required property is an interface property that must be present in an object assigned to that interface type. By default, properties declared in a TypeScript interface are required unless they are explicitly marked optional. TypeScript checks both

the presence of the property and the compatibility of its value. Required properties are useful for information that every object must contain.

Example

```
interface Product {
        name: string;
        price: number;
}
```

Both:

name

price

are required.

Correct

```
const product1: Product = {
        name: "Keyboard",
        price: 1500
};
```

Incorrect

```
const product2: Product = {
        name: "Keyboard"
};
```

The second object is missing the required price property.

Scenario

Every item displayed in an online store must have a product name and a price. These can therefore be required interface properties.

4.5.7 Property Type Checking

Interfaces also check whether property values have compatible types.

Example

```
interface Product {
        name: string;
        price: number;
```

```
}
```

Correct

```
const laptop: Product = {
        name: "Laptop",
        price: 50000
};
```

Incorrect

```
const laptop: Product = {
        name: "Laptop",
        price: "Fifty Thousand"
};
```

TypeScript reports an error because:

price

is expected to be:

number

but a string was supplied.

Scenario

A shopping website may perform:

Price * Quantity

Therefore, defining price as a number helps prevent accidental textual values.

4.5.8 Optional Properties in Interfaces

Definition: An optional property is an interface property that an object may contain but is not required to contain. In TypeScript, an optional property is written using the ? symbol after the property name. Optional properties are useful when some information is available only for certain objects. When accessed, an optional property may have the value undefined if it was not supplied.

Syntax

```
interface InterfaceName {
        requiredProperty: type;
        optionalProperty?: type;
}
```

Example

```
interface Product {
        name: string;
        price: number;
        discount?: number;
}
```

Here:

discount?: number

is optional.

Object without discount

```
const product1: Product = {
        name: "Laptop",
        price: 50000
};
```

Object with discount

```
const product2: Product = {
        name: "Mouse",
        price: 800,
        discount: 10
};
```

Both objects satisfy the interface.

4.5.9 Scenario - Optional Product Discount

Some products may have discounts while others may not.

```
interface Product {
        name: string;
        price: number;
        discount?: number;
}
```

This structure represents:

name

→ Required

price

→ Required

discount

→ Optional

The website can therefore store both discounted and non-discounted products using the same interface.

4.5.10 Readonly Properties

Definition: A readonly property is an interface property that cannot be reassigned through that typed reference after the object has been initialized. It is useful for values that should not normally change after creation. The readonly modifier improves protection against accidental reassignment. It applies to TypeScript's type checking rather than making all nested data automatically immutable.

Example

```
interface Product {
        readonly productId: number;
        name: string;
        price: number;
}
```

Object:

```
const laptop: Product = {
        productId: 101,
        name: "Laptop",
        price: 50000
};
```

The following reassignment is not allowed:

```
laptop.productId = 102;
```

Scenario

A product ID should normally remain unchanged after the product has been created.

4.5.11 Interfaces with Methods

Definition: An interface can describe methods in addition to data properties. A method declaration specifies the method name, parameters, and return type that an object is expected to provide. The interface describes the required method structure but does not normally provide the method implementation itself. Objects implementing that structure provide the actual behaviour.

Syntax

```
interface InterfaceName {
        methodName(): returnType;
}
```

Example

```
interface Product {
        name: string;
        price: number;
        displayProduct(): void;
}
```

Object:

```
const laptop: Product = {
        name: "Laptop",
        price: 50000,

        displayProduct(): void {
                console.log(
                        this.name +
                        " - " + ₹
                        this.price
                );
        }
};
```

Function call:

```
laptop.displayProduct();
```

Output:

Laptop – 50000 ₹

Scenario

A product object can contain both product information and a method for displaying that information.

4.5.12 Interface as a Function Parameter Type

Interfaces are very useful when a function receives an object.

Example

```
interface Product {
        name: string;
        price: number;
}
```

Function:

```
function displayProduct(
        product: Product
): void {
        console.log(
                product.name +
                " – " + ₹
                product.price
        );
}
```

Object:

```
const laptop: Product = {
        name: "Laptop",
        price: 50000
};
```

Call:

```
displayProduct(laptop);
```

Output:

Laptop – 50000 ₹

Working

Product Object

↓

Passed to Function

↓

Interface Defines Expected Structure

↓

Function Uses Properties

Scenario

A product-display function can receive any product object that satisfies the required Product structure.

4.5.13 Interface with Different Objects

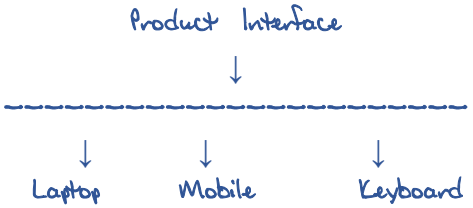
One interface can describe many compatible objects.

```
interface Product {
        name: string;
        price: number;
}
```

Objects:

```
const laptop: Product = {
        name: "Laptop",
        price: 50000
};
const mobile: Product = {
        name: "Mobile",
        price: 20000
};
const keyboard: Product = {
        name: "Keyboard",
        price: 1500
};
```

All three objects share the same structure.



4.5.14 Duck Typing

Definition: Duck typing is an informal term used to describe compatibility based primarily on the properties and behaviours that a value provides rather than only on its declared name or origin. TypeScript more precisely uses a structural type system, meaning that an object can be compatible with an interface when it contains the required members with compatible types. The object does not necessarily need to be explicitly declared using that interface. This allows flexible reuse of objects that share the required structure.

Note: If an object has the required structure, TypeScript can treat it as compatible.

4.5.15 Structural Typing in TypeScript

Definition: Structural typing is a type compatibility system in which types are compared according to the members they contain. If a value has the required properties and methods with compatible types, it can often be used where that structure is expected. TypeScript therefore focuses mainly on the shape of the value rather than requiring an explicit nominal relationship between named types. This behaviour is closely related to what programmers commonly call duck typing.

Basic Example

```
interface Product {
        name: string;
        price: number;
}
```

Create an object without explicitly writing : Product:

```
const item = {
        name: "Laptop",
        price: 50000,
        stock: 10
};
```

Function:

```
function showProduct(
    product: Product
): void {
    console.log(
        product.name
    );
}
```

Call:

```
showProduct(item);
```

This is accepted because item contains at least the required compatible members:

name → string

price → number

4.5.16 Duck Typing Example

Consider:

```
interface Student {
    name: string;
    department: string;
}
```

Function:

```
function showStudent(
    student: Student
): void {
    console.log(
        student.name +
        " - " +
        student.department
    );
}
```

Object:

```
const person = {
    name: "Ananya",
    department: "CSE",
    age: 18
};
```

Call:

```
showStudent(person);
```

Output:

Ananya – CSE

The object contains an extra property:

age

but it still contains all members required by Student.

Working

Required by Student:

name

department

Object Contains:

name

department

age

Required Structure Present

↓

Compatible

4.5.17 Duck Typing Does Not Mean "Any Object"

Students should understand an important restriction.

Suppose:

```
interface Product {
    name: string;
    price: number;
```

}

Object:

```
const item = {
        title: "Laptop",
        cost: 50000
};
```

Function:

```
function displayProduct(
        product: Product
): void {

        console.log(product.name);
}
```

The following is not compatible:

```
displayProduct(item);
```

because the required properties:

name

price

are missing.

Therefore:

Duck / Structural Typing

does NOT mean

"any object is accepted"

It means:

Required compatible structure

must be present.

4.5.18 Extra Properties

Suppose the interface requires:

```
interface Product {
        name: string;
```

```
    price: number;
}
```

An existing variable can contain additional information:

```
const item = {
    name: "Laptop",
    price: 50000,
    stock: 10
};
```

Passing the variable:

displayProduct(item); can be compatible because the required properties are available.

remember:

Interface Requirements

↓

Must Be Present

Additional Compatible Information

↓

May Exist

TypeScript also performs additional excess property checks in some contexts, especially when a fresh object literal is assigned directly to an interface type. That finer rule can be studied later; the important beginner concept is structural compatibility.

4.5.19 Interface Extending Another Interface

Definition: An interface can extend another interface to reuse an existing structure and add additional members. The extends keyword creates a relationship in which the new interface includes the members of the existing interface. This reduces repetition when several related object structures share common properties. The extending interface can then add more specialized properties.

Example

```
interface Product {
    name: string;
    price: number;
```

```
}
```

Extended interface:

```
interface DigitalProduct
        extends Product {
        fileSize: string;
}
```

Object:

```
const ebook: DigitalProduct = {
        name: "TypeScript eBook",
        price: 500,
        fileSize: "20 MB"
};
```

Working



Scenario

An e-commerce website may have normal products and digital products. A digital product can reuse common product properties and add a file size.

4.5.20 Interface vs Object

Students should not confuse an interface with an actual object.

Interface	Object
Describes expected structure	Contains actual values
Used mainly for type checking	Exists at runtime
Does not itself store product data	Stores product data
Example: interface Product	Example: const laptop = {...}

Example

Interface:

```
interface Product {
        name: string;
        price: number;
}
```

Object:

```
const laptop: Product = {
        name: "Laptop",
        price: 50000
};
```

Simple memory:

Interface

→ Structure

Object

→ Actual Data

4.5.21 Interface and Class

Interfaces and classes may appear similar because both can describe members, but they are different concepts.

Interface	Class
Describes expected structure	Defines structure and implementation
Mainly used for type checking	Can create objects
Does not use new to create an interface object	Objects can be created using new
Does not normally provide runtime implementation	Can contain constructors and implemented methods

For this topic, mainly need to remember:

Interface

→ Describes what should exist

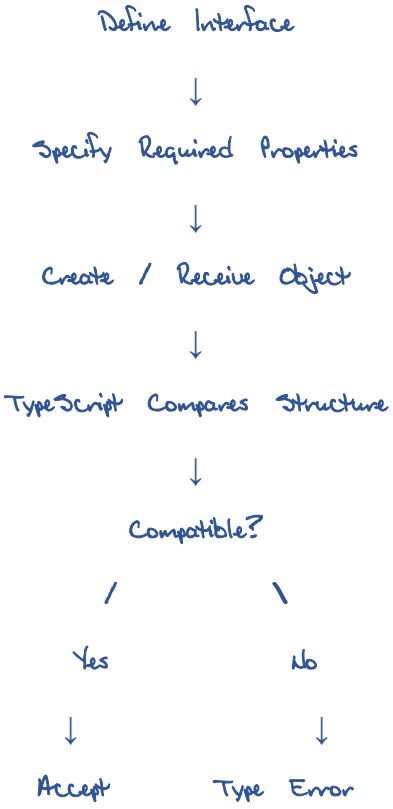
Class

→ Can define how objects are created and behave

Classes are studied in detail in Topic 4.10.

4.5.22 Working Principle of Interfaces

The basic process can be represented as:



4.5.23 Scenario - Product Catalogue

Consider an online store.

Every product requires:

Product Name

Price

Availability

Some products may additionally have a discount.

An interface can define:

```
interface Product {
        name: string;
        price: number;
        available: boolean;
```

```
    discount?: number;
}
```

Objects:

```
const laptop: Product = {
    name: "Laptop",
    price: 50000,
    available: true
};
```

and:

```
const mouse: Product = {
    name: "Wireless Mouse",
    price: 800,
    available: true,
    discount: 10
};
```

Both satisfy the same interface.

4.5.24 Program

The following program demonstrates an interface, required properties, an optional property, a function using an interface, and structural typing.

```
interface Product {
    name: string;
    price: number;
    available: boolean;
    discount?: number;
}
function displayProduct(
    product: Product
): void {
    console.log(
        "Product: " +
```

```
            product.name
        );
        console.log(
            "Price: " + ₹
            product.price
        );
        console.log(
            "Available: " +
            product.available
        );
        if (
            product.discount !== undefined
        ) {
            console.log(
                "Discount: " +
                product.discount +
                "%"
            );
        }
}
const laptop: Product = {
        name: "Laptop",
        price: 50000,
        available: true
};
const mouse = {
        name: "Wireless Mouse",
        price: 800,
        available: true,
        discount: 10,
        stock: 25
```

```
};

displayProduct(laptop);

console.log("------");

displayProduct(mouse);
```

4.5.25 Program Output

```
Product: Laptop

Price: 50000 ₹

Available: true

------

Product: Wireless Mouse

Price: 800 ₹

Available: true

Discount: 10%
```

4.5.26 Program Explanation

The interface:

interface Product

defines the expected structure.

The properties:

name: string;

price: number;

available: boolean;

are required.

The property:

discount?: number;

is optional.

The function:

```
function displayProduct(
        product: Product
```

): void

can receive objects compatible with the Product interface.

The laptop object is explicitly declared as:

const laptop: Product

The mouse object is not explicitly declared using Product.

However, it contains:

name

price

available

discount

with compatible types.

It also contains:

stock

as additional information.

When the variable mouse is passed to:

displayProduct(mouse);

TypeScript can accept it because it contains the required Product structure.

This demonstrates structural typing, commonly explained using the idea of duck typing.

4.5.27 Interface and Duck Typing Relationship

The relationship can be remembered as:

Interface

→ Defines Required Structure

Object

→ Provides Actual Members

Structural / Duck Typing

→ Checks Whether Required Members Exist

Example:

Product Interface requires:

name

price

↓

Object contains:

name

price

stock

↓

Required members available

↓

Compatible

4.5.28 Important Points

An interface is declared using: interface

Example:

```
interface Product {
        name: string;
        price: number;
}
```

A required property is written normally:

price: number;

An optional property uses: ?

Example:

discount?: number;

An interface can also describe methods.

TypeScript mainly uses structural typing.

The easiest memory rule is: Interface

→ Defines Shape

Structural / Duck Typing

→ Checks Shape

4.5.29 Scenario-Based Questions

Question 1

An online shopping application wants every product object to contain a name and price. Which TypeScript feature can describe this structure?

Answer: An interface.

```
interface Product {
        name: string;
        price: number;
}
```

Question 2

A product may or may not contain a discount. How should the discount property be declared?

Answer: As an optional property.

```
discount?: number;
```

Question 3

What does the ? symbol indicate in an interface property?

Answer: It indicates that the property is optional.

Question 4

Consider:

```
interface Student {
        name: string;
        department: string;
}
```

An object contains:

```
const studentData = {
        name: "Ravi",
        department: "CSE",
        age: 18
};
```

Can this object generally be passed to a function expecting Student?

Answer: Yes, because it contains the required compatible name and department properties.

Question 5

What TypeScript concept explains compatibility based on object structure?

Answer: Structural typing, commonly discussed informally as duck typing.

Question 6

Does duck typing mean that every object is automatically accepted?

Answer: No. The object must contain the required members with compatible types.

Question 7

Can an interface contain method declarations?

Answer: Yes.

Example:

```
interface Product {
        displayProduct(): void;
}
```

Question 8

Does an interface itself create an object?

Answer: No. An interface describes an expected structure, an actual object must be created separately.

4.5.3D Important Questions

1. What is an interface in TypeScript?

An interface is a type declaration that describes the expected structure of an object. It can specify properties, methods, and their expected types.

2. Why are interfaces used?

Interfaces provide a common structure for objects and allow TypeScript to check whether supplied values satisfy that structure.

3. What is an optional interface property?

An optional property is a property that an object may omit. It is written using the ? symbol after the property name.

4. Give the syntax of an interface.

```
interface InterfaceName {
        propertyName: type;
}
```

5. Can interfaces contain methods?

Yes. Interfaces can describe the names, parameters, and return types of methods that an object should provide.

6. What is duck typing?

Duck typing is an informal way of describing compatibility based on the members an object provides rather than only on its declared type name.

7. What is structural typing?

Structural typing is the TypeScript type-system behaviour in which compatibility is determined mainly by the structure and compatible members of values.

8. What is the relationship between duck typing and structural typing?

Duck typing is often used informally to explain the practical idea, while structural typing is the more precise term for TypeScript's type compatibility model.

9. Can an object contain additional properties and still satisfy an interface?

In many structural-typing situations, yes, provided the required members are present and compatible. Some direct object-literal assignments are also subject to excess property checking.

10. What is the difference between an interface and an object?

An interface describes the expected structure, while an object contains actual runtime values.

4.5.31 Summary

An interface describes the expected structure of an object in TypeScript. It can specify required properties, optional properties, and methods.

Example:

```
interface Product {
    name: string;
    price: number;
    discount?: number;
}
```

TypeScript mainly follows a structural type system. This means that an object may be compatible with an interface when it contains the required members with compatible types, even if the object was not explicitly declared using that interface.

The fundamental relationship is:

Interface

↓

Defines Required Structure

↓

Object

↓

TypeScript Checks Members

↓

Compatible Structure

↓

Object Can Be Used

4.6 Classes, Constructors and Access Modifiers

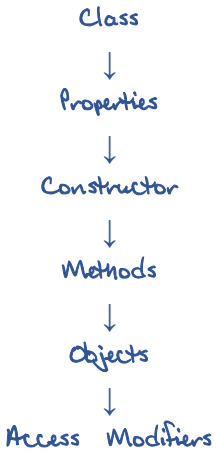
4.6.1 Introduction

Classes provide an organized way to represent related objects in a TypeScript program. Students have already studied the basic idea of JavaScript classes; TypeScript extends that idea by allowing properties, constructor parameters, and methods to use explicit data types.

For example, an online shopping website may need to represent many products. Every product can contain common information such as product name and price and may perform operations such as displaying product details.

TypeScript classes can also control where class data may be accessed. This is achieved using access modifiers such as public, private, and protected.

The basic relationship is:



4.6.2 Class

Definition: A class in TypeScript is a programming construct that defines a common structure for creating objects with related properties and methods. TypeScript classes extend JavaScript classes by allowing properties, parameters, and methods to include type information. Multiple objects can be created from the same class while each object maintains its own values. Classes therefore provide an organized and reusable way to represent similar entities in an application.

Basic Syntax

```
class ClassName {
        propertyName: type;
}
```

Example

```
class Product {
        name: string;
```

}

Here:

Product

→ Class name

name

→ Property

string

→ Property type

Scenario

An online shopping website may use a Product class as the common structure for:

Laptop

Mobile

Keyboard

Mouse

All these products can be represented using objects created from the same class.

4.6.3 Need for Classes

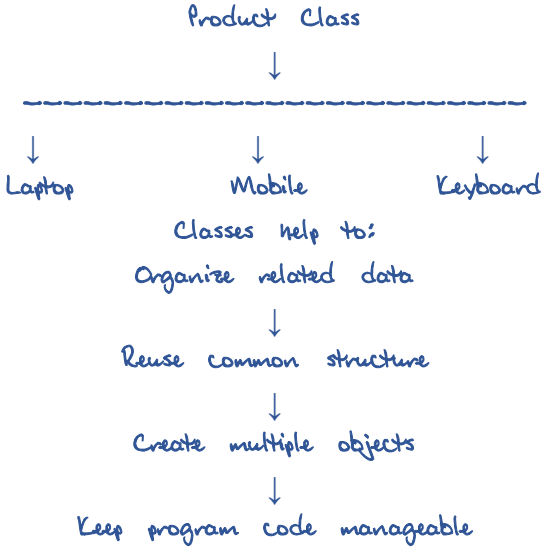
Suppose an online shopping application stores several products separately.

```
const product1 = {
        name: "Laptop",
        price: 50000
};

const product2 = {
        name: "Mobile",
        price: 20000
};
```

As the number of products increases, repeatedly writing similar structures becomes difficult to organize.

A class provides one common design:



Scenario

An e-commerce application containing hundreds of products can define one Product class and create separate objects for individual products.

4.6.4 Declaring a TypeScript Class

Syntax

```
class ClassName {
    property1: type;
    property2: type;
}
```

Example

```
class Product {
    name: string;
    price: number;
}
```

The class declares two properties:

name

→ string

price

→ number

At this stage, the class provides the structure. Values still need to be initialized before the properties are used.

4.6.5 Class Properties

Definition: A property is a variable associated with an object that stores information describing the object's state or characteristics. TypeScript allows the expected type of each class property to be specified explicitly. Every object created from a class can contain its own values for those properties. Properties are commonly used to store information such as names, prices, quantities, marks, and status values.

Example

```
class Product {
        name: string;
        price: number;
}
```

Here: name price are properties.

Property

→ What an object HAS

For a product:

Product HAS:

name

price

stock

Scenario

A shopping website may represent a product using:

name → "Laptop"

price → 50000

These values are stored as object properties.

4.6.6 Object Creation

Definition: An object is an individual instance created from a class and represents a particular entity in a program. Objects created from the same class share the common structure defined by that class but can contain different property values. The new keyword is commonly used to create an object from a TypeScript class. Each object can then use the properties and methods defined by the class.

Syntax

```
const objectName = new ClassName();
```

Example

```
class Product {
}
```

Create an object:

```
const product1 =
        new Product();
```

Here:

Product

→ Class

product1

→ Object

new

→ Creates the object

Scenario

If Product is the class:

product1 → Laptop

product2 → Mobile

product3 → Keyboard

Each object can represent a different item on the website.

4.6.7 Constructor

Definition: A constructor is a special method of a class that executes automatically when a new object is created. It is primarily used to initialize the properties of the newly created object with starting values. TypeScript allows constructor parameters to have explicit data types, helping the compiler check whether correct values are supplied during object creation. A class normally contains one constructor implementation.

Syntax

```
class ClassName {
```

```
        constructor(
            parameter: type
        ) {
            // Initialization
        }
}
```

Example

```
class Product {
        name: string;
        price: number;
        constructor(
            name: string,
            price: number
        ) {
            this.name = name;
            this.price = price;
        }
}
```

Create an object:

```
const product1 =
        new Product(
            "Laptop",
            50000
        );
```

Working

new Product("Laptop", 50000)

↓

Constructor Executes

↓

name receives "Laptop"

price receives 50000

↓

Object Initialized

Scenario

When a shopping website creates a new product, the product name and price can be supplied immediately through the constructor.

4.6.8 Constructor Parameters

Constructor parameters receive the initial values used for an object.

Example

```
constructor(
        name: string,
        price: number
) {
        this.name = name;
        this.price = price;
}
```

Here:

name

→ string parameter

price

→ number parameter

If the programmer writes:

```
new Product(
        "Laptop",
        "Fifty Thousand"
);
```

TypeScript reports a type error because the constructor expects price to be a number.

Scenario

A product-management system can ensure that every newly created product receives a valid textual name and numerical price.

4.6.9 This Keyword

Definition: This keyword refers to the current object when used in the normal instance context of a TypeScript class. It allows the constructor and instance methods to access the properties belonging to that particular object. The same class code can therefore work with different values stored in different objects. It is commonly used to distinguish object properties from constructor parameters having similar names.

Example

```
class Product {
        name: string;
        constructor(
                name: string
        ) {
                this.name = name;
        }
}
```

The statement:

```
this.name = name;
```

means: this.name

→ Property belonging to current object

name

→ Value received through parameter

Scenario

For:

```
const product1 =
        new Product("Laptop");

const product2 =
        new Product("Mobile");
```

the values are:

product1.name → Laptop

product2.name → Mobile

Memory

this

→ Current object

4.6.10 Methods in a Class

Definition: A method is a function defined as part of a class to represent an operation that objects of that class can perform. A method can access class properties through the current object and may receive parameters or return values. TypeScript allows method parameters and return values to have explicit types. Methods are used to place behaviour related to an object together with its data.

Example

```
class Product {
        name: string;
        price: number;
        constructor(
                name: string,
                price: number
        ) {
                this.name = name;
                this.price = price;
        }

        displayProduct(): void {
                console.log(
                        this.name +
                        " - " + ₹
                        this.price
                );
        }
}
```

Create and use an object:

```
const laptop =
        new Product(
                "Laptop",
                50000
        );

laptop.displayProduct();
```

Output

Laptop - 50000 ₹

Understanding

Property

→ What an object HAS

Method

→ What an object CAN DO

Scenario

A product object can contain its own product information and a method that displays that information on a shopping application.

4.10.11 Access Modifiers

Definition: Access modifiers are TypeScript keywords that control where class members can be accessed in a program. They help define which properties and methods should be available outside a class and which should remain restricted. The commonly used TypeScript access modifiers are public, private, and protected. Access modifiers support encapsulation by controlling direct access to class data and behaviour.

The three important modifiers are:

public

private

protected

Class Member

↓

Access Modifier

↓

4.6.12 The public Access Modifier

Definition: The public access modifier allows a class member to be accessed from the class itself, its derived classes, and code outside the class. In TypeScript, class members are public by default when no access modifier is explicitly written. It is suitable for properties and methods that form part of the normal external interface of an object. The keyword may still be written explicitly to make the intended access level clear.

Syntax

```
public propertyName: type;
```

Example

```
class Product {
        public name: string;
        constructor(
                name: string
        ) {
                this.name = name;
        }
}
```

Outside the class:

```
const product1 =
        new Product("Laptop");
console.log(
        product1.name
);
```

Output

Laptop

Because:

name

→ public

→ Can be accessed outside class

Scenario

A product name may need to be displayed throughout a shopping website. It can therefore be declared public.

4.6.13 Default Access in TypeScript

If no access modifier is written, a normal TypeScript class member is public.

For example:

```
class Product {
    name: string;
}
```

is publicly accessible in the same general sense as:

```
class Product {
    public name: string;
}
```

Example:

No Modifier

↓

public by default

Writing public explicitly is sometimes useful when teaching or when the programmer wants the intended access level to be clear.

4.6.14 The private Access Modifier

Definition: The private access modifier restricts a class member so that it can be accessed only within the class that declares it. Code outside the class cannot directly access that member through an object. Private members are useful for protecting internal data and requiring external code to interact with that data through appropriate class methods. This improves encapsulation and reduces accidental modification of internal state.

Syntax

```
private propertyName: type;
```

Example

```
class BankAccount {
    private balance: number;
    constructor(
```

```
        balance: number
    ) {
        this.balance = balance;
    }
    showBalance(): number {
        return this.balance;
    }
}
```

Create object:

```
const account =
    new BankAccount(5000);
```

Allowed:

```
console.log(
    account.showBalance()
);
```

Output

5000

Direct access is not allowed:

```
console.log(
    account.balance
);
```

TypeScript reports an error because balance is private.

Working

balance
↓
private
↓
Accessible Inside BankAccount
↓
Not Directly Accessible Outside

Scenario

A payment application may keep an account balance as private data. Other code should use approved methods rather than directly modifying the balance.

4.6.15 Why Use private?

Suppose a shopping application stores a product price.

If price is public:

product.price = -5000;

outside code could directly assign an invalid value.

With a private property, direct external modification can be restricted.

```
class Product {
        private price: number;
}
```

The class can provide a method to access it safely.

```
getPrice(): number {
        return this.price;
}
```

Scenario

An e-commerce application may keep pricing information private and expose only approved operations for reading or changing it.

4.6.16 The protected Access Modifier

Definition: The protected access modifier allows a class member to be accessed within the class that declares it and within classes derived from that class. Unlike a public member, it cannot normally be accessed directly from outside through an ordinary object reference. Unlike a private member, it remains available to child classes. Protected members are useful when related classes need shared internal information that should not be exposed publicly.

Syntax

protected propertyName: type;

Example

```
class User {
        protected username: string;
        constructor(
                username: string
        ) {
```

```
        this.username = username;
    }
}
```

Child class:

```
class Student extends User {
    displayUser(): void {
        console.log(
            this.username
        );
    }
}
```

Create object:

```
const student = new Student("Ananya");
student.displayUser();
```

Output

Ananya

The child class can access:

this.username

because username is protected.

Scenario

A college portal may have a general User class and a specialized Student class. A username may be required by child classes but should not be directly accessible from unrelated outside code.

4.6.17 private and protected Difference

Students commonly confuse these two modifiers.

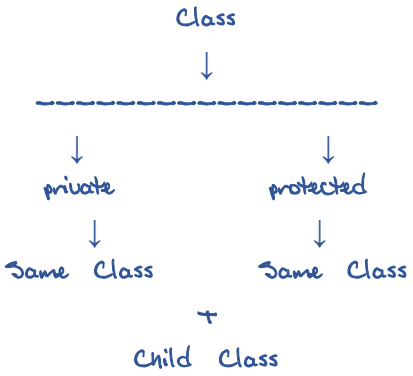
private

→ Same class only

protected

→ Same class + child classes

Example relationship



4.6.18 Public, Private and Protected Comparison

Access Modifier	Same Class	Child Class	Outside Class
public	Yes	Yes	Yes
private	Yes	No	No
protected	Yes	Yes	No

Remember:

public

→ Everywhere

private

→ Same class only

protected

→ Same class + child class

4.6.19 Access Modifiers with Methods

Access modifiers can also be applied to methods.

Example

```
class Login {
        private checkPassword(): boolean {
                return true;
        }
        public loginUser(): void {
```

```
            if (
                this.checkPassword()
            ) {
                console.log(
                    "Login Successful"
                );
            }
        }
}
```

The method:

checkPassword()

is private.

The method:

loginUser()

is public.

Outside code can call:

```
const userLogin = new Login();
userLogin.loginUser();
```

but should not directly call the private method.

Scenario

A login application may expose a public loginUser() operation while keeping internal password-checking logic private.

4.6.2D Parameter Properties

TypeScript provides a shorter syntax in which constructor parameters containing access modifiers can automatically become class properties.

Instead of:

```
class Product {
    public name: string;
    private price: number;
```

```
        constructor(
            name: string,
            price: number
        ) {
            this.name = name;
            this.price = price;
        }
}
```

TypeScript can write:

```
class Product {
        constructor(
            public name: string,
            private price: number
        ) {
        }
}
```

Here:

public name

→ Creates public property

private price

→ Creates private property

This is called a parameter property.

For beginners, the longer form should be understood first because it clearly shows properties, parameters, and assignments.

4.10.21 Class, Constructor and Access Modifier Relationship

The overall relationship is:

Class

↓

Defines Structure

↓

Properties + Methods

↓

Constructor

↓

Initializes Object

↓

Access Modifiers

↓

Control Member Access

4.6.22 Scenario - Online Product

Consider an online store where every product contains:

Product Name

Product Price

Stock

Suppose:

product name may be displayed publicly, product price should be controlled inside the class, stock information may be available to related child classes.

This can be represented using:

name

→ public

price

→ private

stock

→ protected

4.6.23 Program

The following TypeScript program demonstrates a class, constructor, properties, methods, and access modifiers.

```
class Product {

        public name: string;
```

```
        private price: number;
        protected stock: number;
constructor(
                name: string,
                price: number,
                stock: number
        ) {
                this.name = name;
                this.price = price;
                this.stock = stock;
        }
        public displayProduct(): void {
          console.log(
                        "Product: " +
                        this.name
                );
                console.log(
                        "Price: " + ₹
                        this.price
                );
        }
    public getPrice(): number {
                return this.price;
        }
}
class StoreProduct
        extends Product {
        public displayStock(): void {
                console.log(
                        "Stock: " +
                        this.stock
```

```
        );
    }
}
const laptop =
    new StoreProduct(
        "Laptop",
        50000,
        10
    );
laptop.displayProduct();
laptop.displayStock();
console.log(
    "Product Name: " +
    laptop.name
);
console.log(
    "Product Price: " + ₹
    laptop.getPrice()
);
```

4.6.24 Program Output

```
Product: Laptop
Price: 50000 ₹
Stock: 10
Product Name: Laptop
Product Price: 50000 ₹
```

4.6.25 Program Explanation

The class:

```
class Product
```

defines the common structure.

The property:

public name: string;

is public and can therefore be accessed outside the class.

The property:

private price: number;

is private and can be accessed directly only inside the Product class.

The property:

protected stock: number;

can be used inside Product and its child class.

The constructor:

```
constructor(
        name: string,
        price: number,
        stock: number
)
```

receives the initial values.

The statements:

```
this.name = name;
this.price = price;
this.stock = stock;
```

initialize the object's properties.

The method:

displayProduct()

displays the product name and price.

The method:

getPrice()

provides controlled access to the private price.

The class:

```
class StoreProduct
        extends Product
```

is used only to demonstrate the meaning of protected.

It can access:

this.stock

because stock is protected.

Therefore, the program demonstrates:

Class

→ Product

Object

→ laptop

Constructor

→ Initializes values

public

→ name

private

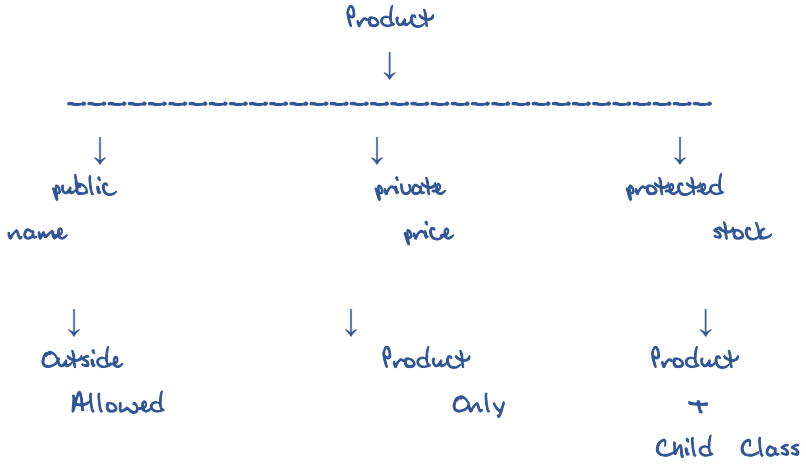
→ price

protected

→ stock

4.6.26 Access Control in the Program

The access relationship can be represented as:



4.6.27 Important Points

A class is declared using:

class

A new object is created using:

new

A constructor is written using:

constructor()

The this keyword normally refers to the current object in class instance code.

Access modifiers are:

public

private

protected

The simplest memory rule is:

Class

→ Common structure

Constructor

→ Initializes object

public

→ Accessible outside

private

→ Same class only

protected

→ Same class + child class

4.6.28 Scenario-Based Questions

Question 1

An online shopping application needs one common structure for creating many products. Which TypeScript feature should be used?

Answer: A class.

class Product { }

Question 2

Which special method automatically executes when a new class object is created?

Answer: The constructor.

constructor() { }

Question 3

Which keyword creates an object from a class?

Answer: new.

const product = new Product();

Question 4

A product name must be accessible from outside the class. Which access modifier is appropriate?

Answer: public.

Question 5

A bank balance should not be accessed directly outside its class. Which access modifier should be used?

Answer: private.

Question 6

A property should be available to its own class and its child classes but not directly outside. Which modifier is suitable?

Answer: protected.

Question 7

What is the default accessibility of a normal TypeScript class member when no modifier is specified?

Answer: public.

Question 8

What does this.name represent inside a class?

Answer: It represents the name property of the current object in the normal instance context.

4.6.29 Important Questions

1. What is a class in TypeScript?

A TypeScript class defines a common structure for creating related objects. It can contain typed properties, constructors, and methods.

2. What is an object?

An object is an individual instance created from a class. Different objects can contain different values while following the same class structure.

3. What is a constructor?

A constructor is a special class method that runs when an object is created. It is normally used to initialize the object's properties.

4. What is the purpose of this?

this allows class instance code to refer to properties and methods belonging to the current object.

5. What are access modifiers?

Access modifiers control where class properties and methods may be accessed.

6. Name the three commonly used TypeScript access modifiers.

They are:

public

private

protected

7. What is public?

A public member can be accessed from inside the class, child classes, and outside code.

8. What is private?

A private member can be accessed directly only within the class that declares it.

9. What is protected?

A protected member can be accessed within its declaring class and derived classes but not normally through an outside object reference.

10. What is the difference between private and protected?

A private member is restricted to the declaring class, whereas a protected member is also accessible within derived classes.

4.6.3D Summary

A class defines a reusable structure for creating objects in TypeScript. It can contain typed properties and methods.

A constructor executes when a new object is created and is commonly used to initialize object properties.

TypeScript provides access modifiers for controlling class-member accessibility:

public

→ Accessible generally

private

→ Accessible only within declaring class

protected

→ Accessible within declaring class and derived classes

4.7 Namespaces and Modules

4.7.1 Introduction

As a TypeScript application becomes larger, keeping all variables, functions, classes, and other program components in one file makes the program difficult to understand and maintain. Different parts of the program may also accidentally use the same names.

For example, an online shopping website may contain separate features for products, shopping carts, users, and payments.

Product Management

Shopping Cart

User Login

Payment

Order Processing

TypeScript provides namespaces and modules to organize related program components.

A namespace groups related declarations under a common name, while a module normally organizes code into separate files and shares selected components using export and import.

4.7.2 Need for Code Organization

Consider a large shopping website containing:

Product functions

Cart functions

Login functions

Payment functions

Order functions

If everything is placed in one file:

app.ts

↓

Thousands of lines

↓

Difficult to understand

↓

Possible naming conflicts

↓

Difficult maintenance

A better organization is:

Product Code

↓

Separate Group / File

Cart Code

↓

Separate Group / File

Login Code

↓

Separate Group / File

TypeScript provides:

Namespaces

→ Group related declarations

Modules

→ Organize and share code between files

Scenario

An online store may separate:

Product Operations

Cart Operations

Payment Operations

instead of writing all operations in the same section of code.

4.7.3 Namespace

Definition: A namespace in TypeScript is a language construct used to group related variables, functions, classes, interfaces, and other declarations under a common name. It helps organize program components and can reduce naming conflicts between different parts of an application. Members that need to be accessed outside the namespace can be made available using the export keyword. Namespaces are mainly associated with certain script-based or older TypeScript organization patterns, while modern applications commonly use file-based modules.

Basic Syntax

```
namespace NamespaceName {
    // Related declarations
}
```

Example

```
namespace College {
        const collegeName: string =
                "ABC Engineering College";
}
```

Here:

College

→ Namespace name

The namespace provides a logical group for college-related code.

Scenario

A college portal may group student-related functions inside:

StudentPortal

and admission-related functions inside:

AdmissionPortal

This reduces confusion between different program sections.

4.7.4 Need for a Namespace

Suppose two sections of a website contain functions with the same name.

```
function showDetails(): void {
        console.log("Student Details");
}
```

Another part may also require:

```
function showDetails(): void {
        console.log("Product Details");
}
```

Using the same name in the same scope can create conflicts.

Namespaces allow the functions to be grouped separately.

```
namespace Student {
        export function showDetails(): void {
                console.log("Student Details");
        }
```

```
}
```

and:

```
namespace Product {
        export function showDetails(): void {
                console.log("Product Details");
        }
}
```

Now they can be distinguished as:

```
Student.showDetails();
Product.showDetails();
```

Same Function Name
↓
Possible Conflict
↓
Namespaces
↓
Separate Logical Groups

4.7.3 Declaring Members Inside a Namespace

A namespace can contain different TypeScript declarations.

For example:

```
namespace Store {
        const storeName: string =
                "ABC Online Store";
        function welcome(): void {
                console.log(
                        "Welcome to Store"
                );
        }
}
```

Both declarations belong to the Store namespace.

However, declarations are not automatically available outside the namespace. Members that should be externally accessible are normally exported.

4.7.6 The export Keyword in a Namespace

Definition: The export keyword makes a declaration inside a TypeScript namespace accessible from outside that namespace. Without export, a declaration remains internal to the namespace and cannot normally be accessed using the namespace name from external code. Variables, functions, classes, and other declarations can be exported when they form part of the namespace's public functionality. This provides control over which members are exposed.

Syntax

```
namespace NamespaceName {
    export function functionName(): void {
    }
}
```

Example

```
namespace College {
    export function showCollege(): void {
        console.log(
            "ABC Engineering College"
        );
    }
}
```

Outside the namespace:

```
College.showCollege();
```

Output

ABC Engineering College

Working

Namespace

↓

Member

↓

export

↓

Available Outside Namespace

Scenario

A college portal may export only the functions that other parts of the application need to call.

4.7.7 Namespace Member Access

Exported members are commonly accessed using:

NamespaceName.memberName

Example

```
namespace Shopping {
        export function showCart(): void {
                console.log(
                        "Shopping Cart"
                );
        }
}
```

Call:

```
Shopping.showCart();
```

Output

Shopping Cart

Here:

Shopping

→ Namespace

showCart

→ Exported member

4.7.8 Namespace with Variables

A namespace can also contain exported variables.

Example

```
namespace Website {
        export const websiteName: string = "ABC Learning Portal";
```

```
}
```

Access:

```
console.log (Website.websiteName);
```

Output

ABC Learning Portal

Scenario

A web application may group related configuration values under one namespace.

4.7.9 Namespace with a Class

A namespace may also contain a class.

Example

```
namespace Store {
        export class Product {
                constructor(
                        public name: string,
                        public price: number
                ) {
                }
        }
}
```

Create an object:

```
const laptop =
        new Store.Product(
                "Laptop",
                50000
        );
```

Display:

```
console.log(
        laptop.name
);
```

Output

Laptop

Scenario

An e-commerce application may group product-related classes inside a Store namespace.

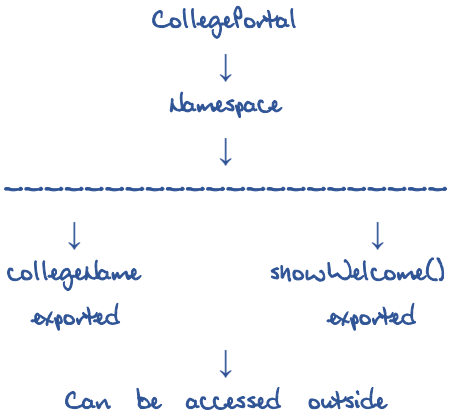
4.7.10 Complete Namespace Example

```
namespace CollegePortal {
        export const collegeName: string =
                "ABC Engineering College";
        export function showWelcome(): void {
                console.log(
                        "Welcome to " +
                        collegeName
                );
        }
}
CollegePortal.showWelcome();
```

Output

Welcome to ABC Engineering College

Working



4.7.11 Module

Definition: A module is a file containing program code that explicitly exports or imports values such as variables, functions, classes, or other components. Modules allow a large TypeScript application to be divided into smaller files with clearly defined

responsibilities. Selected declarations can be shared using export, while another module can use them through import. File-based modules are the standard approach for organizing code in modern TypeScript and JavaScript applications.

Instead of:

app.ts

↓

5000 lines

use:

product.ts

cart.ts

login.ts

payment.ts

app.ts

Each file can contain code related to one specific feature.

Scenario

An online store may organize its program as:

product.ts

→ Product operations

cart.ts

→ Shopping-cart operations

payment.ts

→ Payment operations

app.ts

→ Main application

4.7.12 Need for Modules

Modules are useful because they allow developers to:

Separate Different Features

↓

Reuse Code

↓

Reduce Repetition

↓

Improve Maintenance

↓

Control What Is Shared

Scenario

Suppose the same Product class is required by:

Product Page

Cart Page

Order Page

Instead of rewriting the class, it can be defined once in a module and imported where required.

4.7.13 Exporting from a Module

Definition: The export keyword in a module makes a selected declaration available for use by other modules. Only the program components that need to be shared have to be exported. This creates an explicit boundary between internal code and reusable code. Variables, functions, classes, interfaces, and other supported declarations can be exported.

Example - product.ts

```
export class Product {
        constructor(
                public name: string,
                public price: number
        ) {
        }
}
```

Here: export class Product makes Product available to another module.

export

→ Share from this module

4.7.14 Importing from a Module

Definition: The import statement brings exported declarations from another module into the current module. It allows the current file to use reusable code that has been defined elsewhere. The imported member name must correspond to an exported declaration unless another supported import form is used. Imports therefore allow applications to connect separately organized files.

Basic Syntax

```
import {
        MemberName
} from "./file";
```

Example - app.ts

```
import {
        Product
} from "./product";
```

Now the imported class can be used:

```
const laptop =
        new Product(
                "Laptop",
                50000
        );
```

Simple Memory

export

→ Share

import

→ Use shared code

4.7.15 Named Export

A declaration exported using its name is commonly called a named export.

product.ts

```
export const storeName: string =
        "ABC Store";
export function showStore(): void {
        console.log(storeName);
}
```

Another file can import both members.

app.ts

```
import {
        storeName,
        showStore
} from "./product";
```

Use:

```
console.log(storeName);
showStore();
```

4.7.16 Exporting Multiple Members

Instead of writing export before every declaration, members can also be exported together.

Example

```
const websiteName: string =
        "ABC Store";
function showWelcome(): void {
        console.log(
                "Welcome to " +
                websiteName
        );
}

export {
        websiteName,
        showWelcome
};
```

Another file may import:

```
import {
        websiteName,
        showWelcome
} from "./store";
```

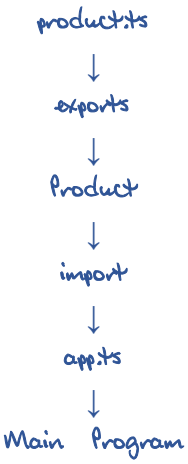
For beginners, either style is acceptable.

4.7.17 Module File Structure

Consider the following files:

```
project/
|
├── product.ts
├── app.ts
└── index.html
```

The relationship is:



This is the fundamental idea of modules.

4.7.18 Simple Two-File Module Example

File 1 - product.ts

```
export class Product {
        constructor(
                public name: string,
                public price: number
        ) {
        }
        displayProduct(): string {
                return (
                        this.name + " - " + ₹
                        this.price
                );
```

```
    }
}
```

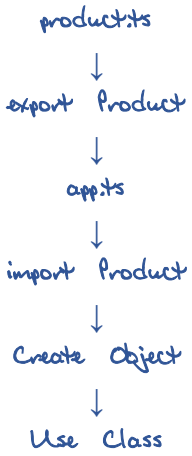
File 2 - app.ts

```
import {
    Product
} from "./product";
const laptop =
    new Product(
        "Laptop",
        50000
    );
console.log(
    laptop.displayProduct()
);
```

Expected Output

Laptop - 50000 ₹

Working



4.7.19 Module and Browser Execution

TypeScript files are compiled into JavaScript before browser execution.

Conceptually:



↓

TypeScript Compiler

↓

product.js

app.js

↓

Browser

When using JavaScript modules directly in a webpage, the main script is loaded as a module.

<script type="module" src="app.js">

</script>

The exact compilation and module configuration can vary between projects. For this syllabus, the important concept is the relationship between:

export

→ Make reusable code available

import

→ Use reusable code

4.7.20 Scenario - Product and Cart Modules

An online shopping website may contain:

product.ts

→ Product class

cart.ts

→ Cart calculations

app.ts

→ Uses both

product.ts

```
export class Product {
        constructor(
                public name: string,
                public price: number
        ) {
        }
```

```
}
```

cart.ts

```
export function calculateTotal(
        price: number,
        quantity: number
): number {
        return price * quantity;
}
```

app.ts

```
import {
        Product
} from "./product";


import {
        calculateTotal
} from "./cart";
const mouse =
        new Product(
                "Wireless Mouse",
                800
        );
const total =
        calculateTotal(
                mouse.price,
                2
        );
console.log(
        "Product: " +
        mouse.name
);
console.log(
```

```
        "Total: " + ₹
        total
};
```

Output

```
Product: Wireless Mouse
Total: 1600 ₹
```

4.7.21 Namespace vs Module

Namespaces and modules both help organize code, but they are used differently.

Namespace	Module
Groups related declarations	Organizes code into files
Uses namespace	Uses file-based export and import
Members may be accessed using namespace name	Members are imported into another module
Useful in certain script-based designs	Preferred for modern applications
Example: Store.Product	Example: import { Product } ...

Namespace

```
namespace Store {
        export function show(): void {
        }
}
```

Usage:

```
Store.show();
```

Module

store.ts

```
export function show(): void {
}
```

app.ts

```
import {
        show
```

```
} from "./store";
```

4.7.22 Important Difference

The easiest distinction is:

Namespace

→ Organizes names inside a logical group

Module

→ Organizes reusable code across files

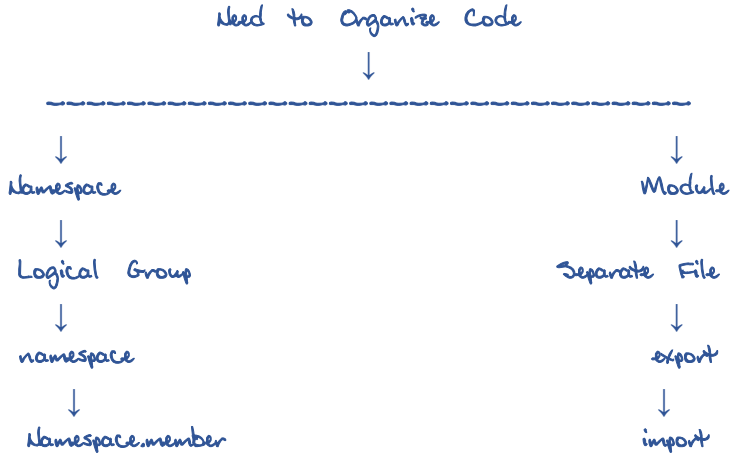
In modern TypeScript development:

Modules

→ Usually preferred for application code

Namespaces are still useful to understand because they are part of the TypeScript language and appear in existing code.

4.7.23 Namespaces and Modules Working Principle



4.7.24 Program - Namespace

The following program demonstrates a simple TypeScript namespace.

```
namespace Shopping {
    export const storeName: string =
        "ABC Online Store";
    export class Product {
        constructor(
            public name: string,
```

```
                public price: number
            ) {
            }
            displayProduct(): void {
                console.log(
                    "Product: " +
                    this.name
                );
                console.log(
                    "Price: " + ₹
                    this.price
                );
            }
        }
        export function showStore(): void {
            console.log(
                "Store: " +
                storeName
            );
        }
}
Shopping.showStore();

const laptop =
    new Shopping.Product(
        "Laptop",
        50000
    );
laptop.displayProduct();
```

4.7.25 Program Output

Store: ABC Online Store

Product: Laptop

Price: 50000 ₹

4.7.26 Program Explanation

The statement:

namespace Shopping

creates a namespace named Shopping.

The declaration:

export const storeName

makes the variable accessible outside the namespace.

The class:

export class Product

is also exported.

Therefore:

Shopping

→ Namespace

storeName

→ Exported variable

Product

→ Exported class

showStore()

→ Exported function

4.7.27 Program - Module

Assume:

product.ts

app.ts

product.ts

```
export class Product {
    constructor(
        public name: string,
        public price: number
    ) {
    }
    displayProduct(): string {
        return (
            this.name +
            " - " + ₹
            this.price
        );
    }
}
```

app.ts

```
import {
    Product
} from "./product";
const product1 =
    new Product(
        "Wireless Keyboard",
        1500
    );
console.log(
    product1.displayProduct()
);
```

Output

```
Wireless Keyboard - 1500 ₹
```

4.7.28 Module Program Explanation

The file:

product.ts

contains the reusable Product class.

The statement:

export class Product

makes that class available to other modules.

The file:

app.ts

uses:

```
import {
        Product
} from "./product";
```

to bring the class into the current module.

The program then creates:

```
new Product(
        "Wireless Keyboard",
        1500
)
```

and uses the imported class.

The complete relationship is:

product.ts
↓
export
↓
Product
↓
import
↓
app.ts
↓
Create and Use Product

4.7.29 Important Points

A namespace is created using:

namespace

Example:

namespace College {

}

A namespace member that must be available outside uses:

export

A module is generally a separate file that exports or imports code.

To share from a module:

export

To use from another module:

import

The simplest memory rule is:

Namespace

→ Group related names

Module

→ Organize related files

export

→ Share

import

→ Use

4.7.30 Scenario-Based Questions

Question 1

A TypeScript application contains two functions called showDetails() for different program sections. Which feature can place them in separate logical groups?

Answer: Namespaces.

```
namespace Student {
        export function showDetails(): void {
        }
```

}

Question 2

Which keyword creates a namespace?

Answer: namespace

Question 3

A function inside a namespace must be accessed outside the namespace. Which keyword should be used?

Answer: export.

Question 4

A shopping application wants to store its Product class in one file and use it in another. Which concept should be used?

Answer: Modules.

Question 5

Which keyword makes a class available from a module?

Answer: export.

```
export class Product {
}
```

Question 6

Which statement brings an exported Product class into another module?

Answer:

```
import {
        Product
} from "./product";
```

Question 7

What is the main difference between a namespace and a module?

Answer: A namespace groups related declarations under a common name, while a module commonly organizes reusable code across separate files using export and import.

Question 8

Which approach is normally preferred for modern TypeScript applications?

Answer: File-based modules are generally preferred.

4.7.31 Important Questions

1. What is a namespace in TypeScript?

A namespace is a TypeScript construct used to group related program declarations under a common name. It can contain variables, functions, classes, and other declarations.

2. Why are namespaces used?

Namespaces help organize related code and reduce naming conflicts between different program sections.

3. What is the purpose of export inside a namespace?

It makes a namespace member accessible from outside the namespace.

4. How is an exported namespace member accessed?

It is commonly accessed using:

NamespaceName.memberName

5. What is a module?

A module is a file that organizes program code and explicitly shares or uses values through mechanisms such as export and import.

6. Why are modules required?

Modules divide large applications into smaller reusable files, improve organization, and reduce code duplication.

7. What is export in a module?

export makes selected declarations available to other modules.

8. What is import?

import brings exported declarations from another module into the current module.

9. What is the difference between export and import?

export shares code from a module, while import brings shared code into another module.

10. What is the difference between namespaces and modules?

Namespaces organize declarations under a common logical name, whereas modules mainly organize and share code across files.

4.7.32 Summary

TypeScript provides namespaces and modules for organizing larger programs.

A namespace groups related declarations under a common name.

Example:

```
namespace College {
        export function show(): void {
        }
}
```

An exported namespace member can be accessed using:

```
College.show();
```

A module normally represents code organized in a separate file.

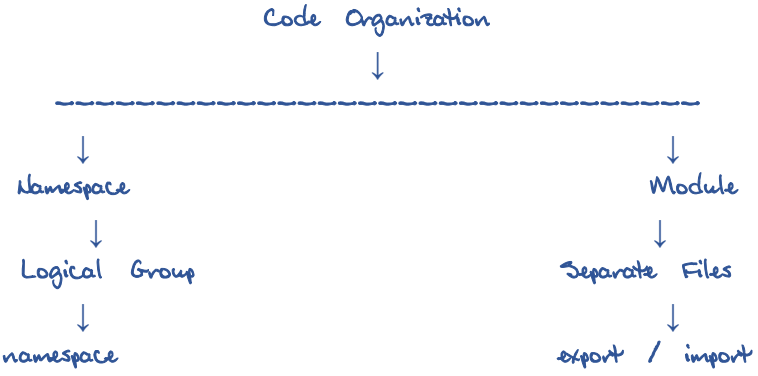
Example:

```
export class Product {   }
```

Another module can use:

```
import {
        Product
} from "./product";
```

The fundamental relationship is:



4.8 Generics and Generic Constraints

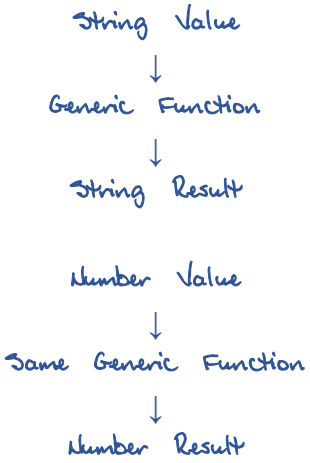
4.8.1 Introduction

In many applications, the same operation may need to work with different types of data. For example, a function may need to return a student name as a string in one situation and a student roll number as a number in another situation.

Writing a separate function for every data type causes unnecessary repetition.

TypeScript provides generics to solve this problem. Generics allow programmers to write reusable functions, interfaces, and classes that can work with different data types while still preserving type information.

For example:



4.8.2 Need for Generics

Consider two functions:

```
function returnNumber(
        value: number
): number {
        return value;
}
```

and:

```
function returnString(
        value: string
): string {
        return value;
}
```

Both functions perform exactly the same operation.

The only difference is the data type.

This results in:

Same Logic
↓
Different Data Types
↓
Repeated Functions

A generic function allows the same logic to work with different types.

```
function returnValue<T>(
        value: T
): T {
        return value;
}
```

Now the same function can work with:

number

string

Boolean and other suitable types

Scenario

A website utility may need to return different kinds of information such as:

Student Name

→ string

Student ID

→ number

Admission Status

→ boolean

A generic function can process each value without creating a separate function for each type.

4.8.3 Generics

Definition: Generics are a TypeScript feature that allows functions, classes, interfaces, and other type structures to work with different data types while preserving information

about those types. A generic declaration uses one or more type parameters that act as placeholders for actual types supplied or inferred when the declaration is used. Generics reduce repeated code because the same program structure can operate safely on multiple types. They therefore provide both code reuse and type safety.

Basic Idea

Generic

↓

Reusable Code

↓

Works with Different Types

↓

Preserves Type Information

4.8.4 Type Parameter

Definition: A type parameter is a placeholder used in a generic declaration to represent a type that will be determined when the generic code is used. It is written inside angle brackets such as <T>. The name T is commonly used to mean "type," although other meaningful names may also be used. The same type parameter can be used for parameters, return values, properties, and other type positions within the generic declaration.

Syntax

<T>

The letter:

T

represents a type that will be supplied or inferred later.

Example

```
function showValue<T>(
        value: T
): T {
        return value;
}
```

Here:

<T>

→ Declares type parameter

value: T

→ Parameter uses that type

: T

→ Function returns the same type

4.8.5 First Generic Function

Definition: A generic function is a function that contains one or more type parameters and can therefore operate on values of different types. The type parameter connects the input and output types so that TypeScript can preserve information about the actual type being used. A generic function provides reusable functionality without requiring the programmer to create separate versions for each data type.

Syntax

```
function functionName<T>(
        value: T
): T {
        return value;
}
```

Example

```
function displayValue<T>(
        value: T
): T {
        return value;
}
```

Using a number:

```
console.log(
        displayValue<number>(100)
);
```

Output

100

Using a string:

```
console.log(
```

```
displayValue<string>(
        "TypeScript"
)
);
```

Output

TypeScript

Scenario

A college portal may use the same generic utility function for either a student number or a student name.

4.8.6 Supplying a Type Argument

When a generic function is called, the required type can be written inside angle brackets.

Example

```
displayValue<number>(100);
```

Here:

number

→ Actual type supplied for T

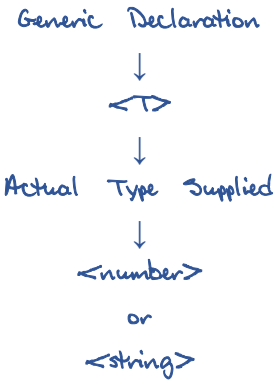
For:

```
displayValue<string>(
        "Laptop"
);
```

the type parameter becomes:

T → string

Therefore:



4.8.7 Type Inference with Generics

TypeScript can often determine the generic type automatically from the argument.

Instead of:

displayValue<number>(100);

the programmer can write:

displayValue(100);

TypeScript infers:

T → number

Similarly:

displayValue("Laptop");

allows TypeScript to infer:

T → string

Scenario

A shopping application passes a product name to a generic function.

```
const result =
        displayValue(
                "Wireless Mouse"
        );
```

TypeScript can infer that the value is a string.

4.8.8 Generic Function with Arrays

Generics can also be used with arrays.

Example

```
function getFirstItem<T>(
        items: T[]
): T {
        return items[0];
}
```

Using numbers:

```
const firstMark =
```

```
    getFirstItem<number>(
        [80, 90, 75]
    );
console.log(firstMark);
```

Output

80

Using strings:

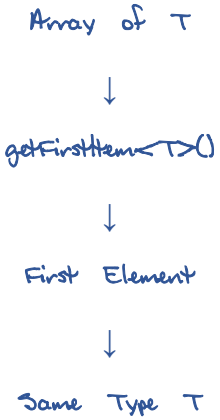
```
const firstCourse =
    getFirstItem<string>(
        [
            "HTML",
            "CSS",
            "TypeScript"
        ]
    );

console.log(firstCourse);
```

Output

HTML

Working



Scenario

A shopping website may use the same function to retrieve the first product name from a string array or the first product price from a number array.

4.8.9 Why Generics Instead of any?

TypeScript also provides the any type, but any removes much of TypeScript's type checking.

Example

```
function returnValue(
        value: any
): any {
        return value;
}
```

The relationship between the input and output type is not preserved safely.

With a generic:

```
function returnValue<T>(
        value: T
): T {
        return value;
}
```

TypeScript knows that the return type is connected to the argument type.

Comparison

any	Generic <T>
Accepts different values	Accepts different types
Loses much type information	Preserves type information
Less type-safe	Maintains stronger type checking
Does not connect input and output types	Can connect input and output types

any

→ Any value, weak type information

Generic

→ Flexible but type-aware

4.8.10 Generic Interface

Definition: A generic interface is an interface that contains one or more type parameters so that the same interface structure can work with different data types. The actual type is supplied when the interface is used. Generic interfaces are useful when a common object structure is needed but one or more properties may contain different types. They provide reusable type definitions while maintaining type checking.

Syntax

```
interface InterfaceName<T> {
        value: T;
}
```

Example

```
interface Box<T> {
        value: T;
}
```

Number object:

```
const numberBox: Box<number> = {
        value: 100
};
```

String object:

```
const textBox: Box<string> = {
        value: "TypeScript"
};
```

Scenario

A web application may use the same response structure to hold different kinds of values.

Box<number>

→ Numeric value

Box<string>

→ Textual value

4.8.11 Generic Interface Website Example

Consider:

```
interface Result<T> {
        data: T;
        success: boolean;
}
```

A student-name result:

```
const studentResult:
        Result<string> = {
        data: "Ananya",
        success: true
};
```

A marks result:

```
const markResult:
        Result<number> = {
        data: 85,
        success: true
};
```

The same interface works with different data types.

Working



4.8.12 Generic Class

Definition: A generic class is a class that uses a type parameter so that objects created from the class can store or process different data types. The type parameter is supplied when an object or class type is used. Members of the class can use that type parameter for properties, constructor parameters, and methods. Generic classes reduce the need to create separate classes for similar structures containing different types.

Syntax

```
class ClassName<T> {
    value: T;
    constructor(
        value: T
    ) {
        this.value = value;
    }
}
```

Example

```
class Box<T> {
    value: T;
    constructor(
        value: T
    ) {
        this.value = value;
    }
}
```

Number object:

```
const numberBox =
    new Box<number>(100);
```

String object:

```
const textBox =
    new Box<string>(
        "Web Programming"
    );
```

Scenario

A web application may create a reusable container for textual messages, numerical values, or other typed information.

4.8.13 Generic Class with a Method

A generic class can also contain methods using the type parameter.

Example

```
class DataStore<T> {
        value: T;
        constructor(
                value: T
        ) {
                this.value = value;
        }
        getValue(): T {
                return this.value;
        }
}
```

Using:

```
const productStore =
        new DataStore<string>(
                "Laptop"
        );
console.log(
        productStore.getValue()
);
```

Output

Laptop

Scenario

An application can use the same DataStore class to store a product name, product number, or another type while preserving its actual type.

4.8.14 Multiple Type Parameters

A generic declaration may use more than one type parameter when different values need different types.

Example

```
function createPair<T, U>(
        first: T,
        second: U
): void {
        console.log(first);
        console.log(second);
}
```

Call:

```
createPair<string, number>(
        "Laptop",
        50000
);
```

Here:

T → string

U → number

Scenario

A product utility may work with:

Product Name → string

Product Price → number using two generic type parameters.

understanding the basic single type parameter <T> is the main requirement.

4.8.15 Limitation of an Unconstrained Generic

Consider:

```
function showLength<T>(
        value: T
): number {
        return value.length;
}
```

TypeScript cannot safely assume that every possible type T contains a length property.

For example:

string

→ Has length

array

→ Has length

number

→ Does not normally have length

Therefore, the generic type needs a rule that says:

Only accept types containing a length property.

This introduces a generic constraint.

4.8.16 Generic Constraint

Definition: A generic constraint restricts the types that are permitted for a generic type parameter. It allows generic code to remain reusable while guaranteeing that supplied types provide certain required properties or capabilities. In TypeScript, the extends keyword is commonly used to apply a constraint to a type parameter. The compiler then accepts only types that satisfy the specified constraint.

Basic Syntax

```
function functionName< T extends ConstraintType> ( value: T) {
}
```

Basic Idea

Generic <T>

→ Many possible types

Constraint

→ Apply a rule

T extends RequiredStructure

→ Only compatible types accepted

4.8.17 The extends Keyword in Generic Constraints

In a generic constraint: T extends SomeType

means: T must satisfy the required structure of SomeType

It does not mean that a new child class is necessarily being created.

This is different from ordinary class inheritance syntax even though both use the word extends.

Generic extends

→ Restrict permitted types

4.8.18 Generic Constraint Using an Interface

First define the required structure:

```
interface HasLength {
        length: number;
}
```

Now create a generic function:

```
function showLength<   T extends HasLength>(
        value: T
): number {
        return value.length;
}
```

The constraint:

T extends HasLength

means that T must contain:

length: number

4.8.19 Generic Constraint Example with String

Call:

```
console.log(
        showLength(
                "TypeScript"
        )
);
```

Output

10

The string is accepted because a string has a length property.

Working

"TypeScript"

↓

Has length?

↓

Yes

↓

Accepted

↓

Return length

↓

10

4.8.2D Generic Constraint Example with Array

Call:

```
console.log(
    showLength(
        [10, 20, 30]
    )
);
```

Output

3

The array is accepted because arrays have a length property.

Scenario

A learning portal can use one generic function to determine the length of either a course-name string or a list of courses, provided the supplied value has a length property.

4.8.21 Invalid Generic Constraint Example

Suppose the function is:

```
function showLength<T extends HasLength>(
        value: T
): number {
        return value.length;
}
```

Calling:

```
showLength(100);
```

causes a TypeScript type error because a number does not satisfy the required HasLength structure.

Working

100

↓

Has length property?

↓

No

↓

Does not satisfy constraint

↓

Type Error

4.8.22 Scenario - Product Objects with IDs

Suppose a website has different object types, but every object processed by a function must contain an ID.

Define:

```
interface HasId {
        id: number;
}
```

Generic function:

```
function displayId< T extends HasId>(
```

```
    item: T
): void {
    console.log(
        "ID: " + item.id
    );
}
```

Product object:

```
const product = {
    id: 101,
    name: "Laptop",
    price: 50000
};
```

Call:

```
displayId(product);
```

Output

```
ID: 101
```

The function does not require the object to contain only id. It simply requires the object to contain a compatible id property.

4.8.23 Generic and Generic Constraint Difference

Generic	Generic Constraint
Allows different types	Restricts which types are allowed
Written using <T>	Commonly uses T extends ...
Focuses on reuse	Adds a required rule
Type may initially be broad	Type must satisfy specified structure

Generic

```
function returnValue<T>(value: T): T {
    return value;
}
```

Constrained Generic

```
function showLength<T extends HasLength>( value: T): number {
        return value.length;
}
```

Easy Memory

Generic

→ Flexible Type

Constraint

→ Flexible Type with a Rule

4.8.24 Generic Function Working Principle

Define Generic Function

↓

Declare <T>

↓

Call Function

↓

Supply / Infer Actual Type

↓

TypeScript Checks Value

↓

Function Processes Value

↓

Preserves Type Information

For a constrained generic:

Value Supplied

↓

Check Constraint

↓

Satisfies Rule?

/ \

Yes No

↓ ↓

Accept Type Error

4.8.25 Scenario - Product Information

Suppose an online shopping website handles different product objects.

Every product-like object processed by a utility must contain: name

but may contain additional information.

Define:

```
interface HasName {
        name: string;
}
```

Generic function:

```
function showItem< T extends HasName>(
        item: T
): void {
        console.log(
                "Item: " + item.name
        );
}
```

Object 1:

```
const laptop = {
        name: "Laptop",
        price: 50000
};
```

Object 2:

```
const ebook = {
        name: "TypeScript eBook",
        fileSize: "20 MB"
};
```

Both can be used:

```
showItem(laptop);
showItem(ebook);
```

because both satisfy:

name: string

4.8.26 Program

The following program demonstrates a generic function, generic class, and generic constraint using a simple shopping website scenario.

```
function returnValue<T>( value: T): T {
        return value;
}
class ProductBox<T> {
        value: T;
        constructor(
                value: T
        ) {
                this.value = value;
        }
        getValue(): T {
                return this.value;
        }
}
interface HasName {
        name: string;
}
function displayItem< T extends HasName>(
        item: T
): void {
        console.log("Item Name: " + item.name);
}
const productName =     returnValue<string>("Laptop" );
const productPrice =returnValue<number>( 50000 );
console.log("Product: " +productName);
console.log(    "Price: " + ₹productPrice);
const productBox = new ProductBox<string>("Wireless Mouse");
console.log(      "Box Value: " +productBox.getValue());
```

```
const product = {   name: "Keyboard", price: 1500,
        stock: 10
};
displayItem(product);
```

4.8.27 Program Output

```
Product: Laptop
Price: 50000 ₹
Box Value: Wireless Mouse
Item Name: Keyboard
```

4.8.28 Program Explanation

The function:

```
function returnValue<T>(
        value: T
): T
```

is a generic function.

The type parameter:

```
<T>
```

acts as a placeholder for the actual type.

The call:

```
returnValue<string>(
        "Laptop"
);
```

uses:

T → string

The call:

```
returnValue<number>(
        50000
);
```

uses:

T → number

Therefore, the same function works with different types.

The class:

class ProductBox<T>

is a generic class.

The object:

new ProductBox<string>(

"Wireless Mouse"

)

uses string as its type.

The property:

value: T

and method:

getValue(): T

therefore work with strings.

The interface:

interface HasName {

name: string;

}

defines the required structure for the constraint.

The function:

function displayItem< T extends HasName>(item: T)

accepts only values whose types contain a compatible name property.

The object:

const product = {

name: "Keyboard",

price: 1500,

stock: 10

};

contains the required:

name: string

property.

Therefore:

displayItem(product);

is accepted.

The complete relationship is:

Generic Function

→ returnValue<T>

Generic Class

→ ProductBox<T>

Generic Constraint

→ T extends HasName

4.8.29 Important Points

A generic type parameter is commonly written as: <T>

The basic generic function syntax is:

```
function functionName<T>(
        value: T
): T {
        return value;
}
```

A generic class can be written as:

```
class Box<T> {
}
```

A generic interface can be written as:

```
interface Result<T> {
}
```

A generic constraint commonly uses:

T extends RequiredType

The easiest memory rule is:

<T>

→ Type placeholder

Generic

→ Reusable for different types

extends

→ Apply a restriction

Generic Constraint

→ Generic with a required rule

4.8.3D Scenario-Based Questions

Question 1

A website needs one function that can return either a string or a number while preserving the supplied type. Which TypeScript feature is suitable?

Answer: Generics.

```
function returnValue<T>(value: T): T {
        return value;
}
```

Question 2

What does <T> represent in a generic declaration?

Answer: It declares a type parameter that acts as a placeholder for an actual type.

Question 3

A generic function is called as:

returnValue<number>(100);

What type does T represent?

Answer: number.

Question 4

Can TypeScript sometimes determine the generic type automatically?

Answer: Yes. TypeScript can often infer the type from the supplied argument.

Question 5

A function must accept only values containing a length property. Which feature should be used?

Answer: A generic constraint.

T extends HasLength

Question 6

Which keyword is commonly used to create a generic constraint?

Answer: extends.

Question 7

Consider: interface HasLength { length: number; }

Can a string satisfy this constraint?

Answer: Yes, because strings provide a compatible length property.

Question 8

Can a plain number such as 100 satisfy the HasLength constraint?

Answer: No, because a number does not provide the required length property.

4.8.31 Important Questions

1. What are generics in TypeScript?

Generics allow functions, classes, interfaces, and other structures to work with different data types while preserving information about the actual type being used. They improve code reuse without giving up type safety.

2. What is a type parameter?

A type parameter is a placeholder for a type in a generic declaration. It is commonly represented using a name such as T.

3. Give the syntax of a generic function.

function functionName<T>(value: T): T {return value; }

4. Why are generics useful?

Generics reduce repeated code because one function, interface, or class can safely work with different types.

5. What is the advantage of generics over any?

Generics preserve the relationship between the actual input and output types, while any removes much of the compiler's type information.

6. What is a generic class?

A generic class contains one or more type parameters and can therefore create objects that work with different specified types.

7. What is a generic interface?

A generic interface describes a reusable object structure containing one or more type parameters.

8. What is a generic constraint?

A generic constraint restricts which types may be used with a generic type parameter.

9. Which keyword is commonly used for a generic constraint?

The extends keyword.

10. What does T extends HasLength mean?

It means that the actual type used for T must satisfy the structure required by HasLength.

4.8.32 Summary

Generics provide a reusable and type-safe way of working with different data types in TypeScript.

A generic type parameter is commonly represented using:

<T>

For example:

```
function returnValue<T>(
        value: T
): T {
        return value;
}
```

The same function can work with strings, numbers, and other suitable types while preserving type information.

Generic interfaces and classes extend the same idea to reusable object structures and classes.

A generic constraint restricts the values that a generic can accept.

Example

```
interface HasLength {
```

```
    length: number;
}

function showLength< T extends HasLength>(
    value: T
): number {
    return value.length;
}
```

The fundamental relationship is:

Generic

↓

<T>

↓

Reusable for Different Types

↓

Preserves Type Information

With constraints:

Generic

↓

<T>

↓

extends

↓

Required Structure

↓

Only Compatible Types Accepted